Neutrino architecture

Radiance 2 technical specification, document 1

r2-spec

Version 0.2, draft of 2026-10-16

Contents

1 Introduction **4**

1.1 What a Neutrino is . . . 4

1.2 What a Neutrino does . . . 4

1.3 Conventions . . . 5

2 Addresses and ownership **5**

2.1 Global addresses . . . 5

2.2 Owners . . . 6

2.3 Granules and destination ranges . . . 7

2.4 The sync-only space . . . 7

3 Compute follows data **8**

3.1 Untracked tasks . . . 9

4 Task descriptor **11**

4.1 Fields . . . 11

4.2 Opcodes . . . 12

4.3 Size in a message . . . 14

5 Instruction set **14**

5.1 Programs and instructions . . . 14

5.2 `invoke` . . . 14

5.3 `spawn` . . . 15

5.4 `wait` . . . 15

5.5 `version`, `vreset` and `vreset_all` . . . 16

5.6 `cycles`, `load` and `store` . . . 16

5.7 Costs . . . 17
5.8 Assembly text . . . 18

6 Tables **19**
6.1 Task entries and tags . . . 19
6.2 Scoreboard and admission . . . 20
6.3 Execution table . . . 21
6.4 Pending table . . . 21
6.5 Version table . . . 21
6.6 Inbound ref table . . . 22
6.7 Inbound invoke table . . . 22
6.8 Outbound dep table . . . 22
6.9 Outbound invoke table . . . 22
6.10 Ticket bitmap . . . 23
6.11 Full tables . . . 23

7 Dependencies and versions **23**
7.1 Hazards . . . 23
7.2 Keys . . . 24
7.3 Version assignment . . . 24
7.4 Lookup . . . 24
7.5 Adoption . . . 25
7.6 Completion and retire . . . 25
7.7 Version reset . . . 26
7.8 Version inference . . . 26
7.9 Joins and signals . . . 26

8 Task lifecycle **27**
8.1 From invoke to ready . . . 27
8.2 Issue, execution and completion . . . 28
8.3 Stalls and refusals . . . 29
8.4 Errors . . . 29
8.5 Program lifecycle . . . 30

9 Protocols between Neutrinos **30**
9.1 Messages . . . 30
9.2 Channels and stages . . . 30

9.3 Dependency protocol . . . 32
9.4 Invoke protocol . . . 33
9.5 Ticket flow . . . 34
9.6 VersionReset . . . 34
9.7 Hierarchy protocol . . . 35
9.8 Ordering . . . 35
9.9 Progress . . . 36

10 Sample programs **37**
10.1 Layernorm on two SMs . . . 37
10.2 GEMM on one SM . . . 38
10.3 A dependency between two SMs . . . 39
10.4 A barrier . . . 40
10.5 Forwarded invokes and inferred versions . . . 40
10.6 A spawn tree and pulls . . . 41
10.7 Data-dependent control and version resets . . . 41

11 Configuration parameters **42**
11.1 Tables . . . 43
11.2 Costs and rates . . . 43
11.3 Global parameters . . . 44

1 Introduction

This document specifies the **Neutrino**, the task engine of a Radiance 2 node. It gives the address ownership rules, the task descriptor and the instruction set. It gives the tables of a Neutrino, the dependency and version rules, the task lifecycle and the protocols between Neutrinos. It also gives sample programs and the configuration parameters of a Neutrino.

The hierarchy of nodes, the memories, the fabric and the PEs are in document 2 (Section 2). The simulator that implements the Neutrino model, r2-sim, is in document 3 (Section 4).

1.1 What a Neutrino is

Each node of the hierarchy has one Neutrino: the device, each cluster and each SM (Figure 1). A Neutrino has these parts:

- A scalar core (RV64) with orchestration instructions (`nu.*`). It runs the **program** of the node.
- A scratch memory for task descriptors and the stack of the program.
- The tables that track tasks, versions and remote requests (Section 6).
- One issue port to each PE of the node, and completion ports back from the PEs.
- An attachment to the control bus of the node, for the messages to and from other Neutrinos.
- Attachments to the data buses of the node, for loads and stores.

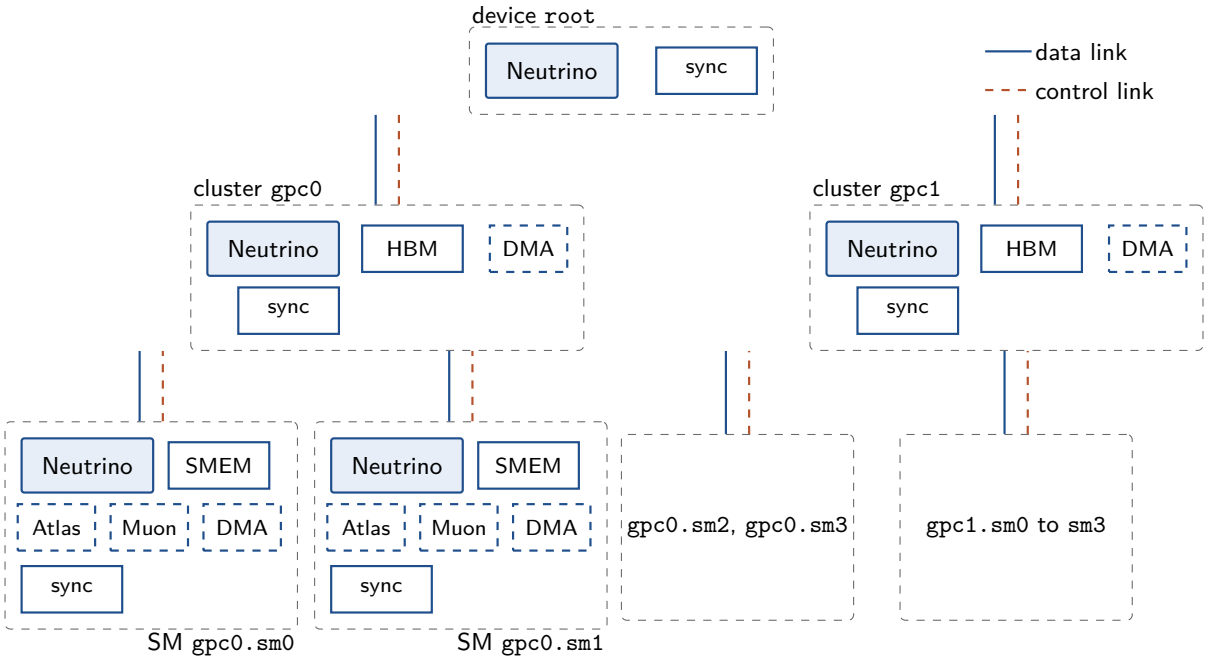


Figure 1: One Neutrino per node in the 2p configuration: the device `root`, the clusters `gpc0` and `gpc1`, and four SMs in each cluster. Each node has a data link and a control link to its parent.

1.2 What a Neutrino does

A Neutrino has these functions:

1. It runs the program of its node. The program builds task descriptors in the scratch memory and submits them with the `invoke` instruction.

2. It holds each task whose destination range it owns, and each untracked task that its program invokes, from the invoke to the retire. A task waits in the scoreboard until its dependencies are done. Then the Neutrino issues the task to a PE of its node.
3. It tracks read-after-write order only. A task names the producers that it reads with keys: an address and a version for versioned data, an address for unversioned data. The owner of the destination range gives each write of a versioned range the next version.
4. It sends and receives the messages of three protocols: the dependency protocol (Await, AwaitAck, Resolve), the invoke protocol (Invoke, VersionReset, and in ticket flow InvokeAck, RetryAck, Reject, PCrdGrant), and the hierarchy protocol (Start, Return).
5. It starts programs on the Neutrinos of its child nodes (`spawn`) and reports the end of its own program to its parent.

The principle of the design is **compute follows data**: a task runs at the Neutrino that owns its destination range, on a PE of that node (Section 3). An invoke whose destination range belongs to another Neutrino travels to that owner as a message (a **forwarded invoke**). A write into the memory of another node is a task of that node: a DMA on its engine that pulls the data. An untracked task is the exception: it runs on a PE of the node that invokes it, and no task can depend on it (Section 3.1).

The Neutrino has no memory allocator. Software decides where each task writes. The hardware tracks no readers: software orders a writer after the readers of a range (write-after-read) and after the earlier writers (write-after-write) with explicit dependencies (Section 7.1).

1.3 Conventions

- The terms are those of the glossary (document 0, Section 4).
- Nodes, memories, PEs and Neutrinos have dotted names. In the 2p configuration the device is `root`, the clusters are `root.gpc0` and `root.gpc1`, and the SMs of cluster g are `root.gpc`g`.sm0` to `root.gpc`g`.sm3`.
- A range is $[base, bound)$. Addresses and lengths are hexadecimal (`0x200`). Sizes use IEC prefixes.
- A cycle is an SM cycle. All costs and latencies are in SM cycles.
- Field names of a descriptor and parameter names are in `typewriter`. Message names are in sans serif.
- The reference configuration is the 2p configuration (document 2, Section 2.5). The values of the examples come from it unless the text names another configuration.
- A default value is the value of a parameter that the examples use when a configuration does not give one. It is not a decision of the architecture.
- Rules are numbered per section: Rule N3.2 is the second rule of Section 3. A rule is normative. A note is informative. An example shows one case.

Note. Document 3 gives the model of the Neutrino in r2-sim (Section 4). The binary encoding of the `nu.*` instructions is not defined yet; Section 5 gives the instructions, their operands and their semantics.

2 Addresses and ownership

2.1 Global addresses

An address is 64 bits wide and global: every Neutrino and every PE uses the same address for the same byte. The high bits are the **prefix**: one field for each level of the hierarchy below the

device. The low bits are the offset. The configuration gives the fields, their widths and the width of the offset (Section 11).

The value of the field of a level selects a child: the child with index i among the children of its parent has the value $i + 1$. The value 0 selects the parent itself. The **own space** of a node is the set of addresses with the prefix of the node and 0 in every field below it. The own space of the device has 0 in every field.

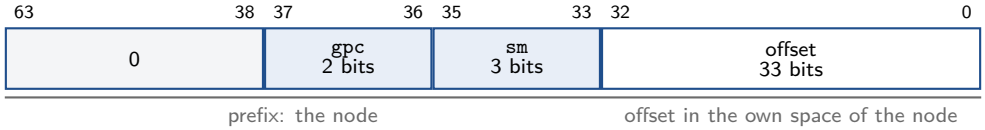


Figure 2: The fields of an address in the 2p configuration. The prefix selects a node. The offset selects a byte in the own space of that node (8 GiB).

Table 1: Own spaces of the nodes in the 2p configuration: an offset of 33 bits, a cluster field of 2 bits and an SM field of 3 bits.

Node	Base of the own space	Example
`root`	`0x0`	`0x0`
`root.gpc`g	$(g+1) \ll 36$	`root.gpc0: 0x10_0000_0000`
`root.gpc`g`.sm`s	$(g+1) \ll 36 \mid (s+1) \ll 33$	`root.gpc0.sm0: 0x12_0000_0000`

2.2 Owners

The **owner** of an address is the Neutrino of the node that holds the address in one of these ranges:

- the range of a scratch memory or a DRAM memory of the node;
- the sync-only range of the node (Section 2.4);
- the linear range of a hash link of the node (document 2, Section 3.3).

Each of these ranges lies in the own space of the node. An address that is in none of these ranges has no owner.

Rule N2.1 (null address). Address 0 is the null address. No range that a node owns may hold it, so no owned range holds granule 0. So the key 0 names no producer: it is the key of the fence (Rule N3.11).

The ownership map is a set of address sets in the sense of Chisel and Rocket: a base and a mask of the free bits. A range whose size is not a power of two, or whose base is not aligned to its size, is the union of the fewest aligned blocks. The address sets of two Neutrinos never overlap. The tables and the protocols see linear addresses only: the rewrite of a hash link applies when a transfer is routed, after the owner is known.

The owner of a task is the owner of its destination range. The owner of a key (Section 7.2) is the owner of the address of the key.

Rule N2.2 (one owner). All granules of the span of a destination range must have one owner. The address of each dependency, of each `wait` except a fence, of each `version` and of each `vreset` must have an owner. A range without one owner causes the error `Unowned`.

2.3 Granules and destination ranges

A Neutrino tracks the ranges that tasks write in **granules**. The parameter `neutrino_granule` gives the size of a granule (Section 11). It is 512 B in the 2p configurations, and 128 B in the RTX PRO 6000 reference configuration, whose kernels write warp tiles with rows of 128 B. All addresses that a Neutrino handles are granule addresses: a byte address divided by the granule. The destination range of a task and the address of a key are granule addresses.

The **destination range** of a task has three fields:

- `base`, 64 bits: the granule address of its first granule;
- `bound`, 32 bits: the number of granules of its span, the granules $[base, base + bound)$;
- `mask`, 32 bits: the granules of the span that the range holds. Granule $base + i$, with $0 \le i < bound$, belongs to the range when $i \,\&\, \neg mask = 0$. A set bit of the mask is a bit of i that can vary, as in the address sets of Rocket. A mask of all ones holds the whole span.

With a mask, destination ranges can interleave: two ranges whose spans overlap can hold disjoint granules, as the example below shows.

Rule N2.3 (granule). The destination range of a task must hold at least one granule: `bound` ≥ 1. A task with `bound` 0 causes the error `Empty`.

Example. A C tile of 128×128 bf16 elements has rows of 256 B. With a granule of 128 B, a row is two granules, and the tile is 256 granules from its base c. Four warps write its four quarters of 64×64 elements. A quarter holds one granule of each of 64 rows: granule $base + i$ of a quarter is in its row $\lfloor i/2 \rfloor$. So each range has the mask `0x7e`, with which bits 1 to 6 of i vary and bit 0 stays 0, and the bound $2 \times 63 + 1 = 127$:

- warp 0: base c: the granules $c, c + 2, \ldots, c + 126$, the left halves of rows 0 to 63;
- warp 1: base $c + 128$: the left halves of rows 64 to 127;
- warp 2: base $c + 1$: the right halves of rows 0 to 63;
- warp 3: base $c + 129$: the right halves of rows 64 to 127.

The spans of warps 0 and 2 overlap, but their ranges hold no common granule. With the bound 127, each span ends at the last granule of its range, so no span goes past the end of the tile. The GEMM program of Section 10.2 uses these ranges.

Note. A key names one granule. A versioned key names the base of a destination range. An unversioned key can name any granule that the destination range of its producer holds; a granule in a gap of the mask does not belong to the range.

2.4 The sync-only space

The **sync-only space** of a node is an address range that the Neutrino tracks for dependencies and that no memory backs. It lies in the own space of the node. A join and a spawn write no data, so their destination ranges are in the sync-only space. They cost only the time of the Neutrino.

Programs use the sync-only space for:

- the destination ranges of joins (signals): many producers to one consumer, read-side order, barriers (Section 7.9);
- the `done` ranges of spawns (Section 5.3);
- other records of completion, for example the store records of the `kernels` program of r2-sim (Section 3.1).

Rule N2.4 (one sync-only range). A node has at most one sync-only range. It must lie in the own space of the node.

Rule N2.5 (signals in the sync-only space). The destination range of a join and the `done` range of a spawn must be in the sync-only space of their owner.

Note. The store records of the `kernels` program of r2-sim are DMA tasks whose destination ranges are in the sync-only space (document 3, Section 13.3.2). The rule does not cover them: it names joins and spawns only.

Example. In the 2p configuration each node has a sync-only range of 16 MiB at the offset `0x1_F000_0000` of its own space. For `root.gpc0.sm0` it starts at `0x13_F000_0000`. Listings print this address as `root.gpc0.sm0.sync+0x0`.

3 Compute follows data

The destination range of a task selects the Neutrino that holds the task and the node whose PE runs it. Data moves to the compute only through copies that the owner of the destination runs. Control moves through messages: a descriptor in an Invoke, a key in an Await, a program in a Start.

An invoke that the fabric routes by address is therefore a dispatch. In a mixture-of-experts layer, the slice of an expert owns a range. The invoke of a task of that expert goes to the Neutrino of the slice, and the activations of the tokens move to the weights of the expert.

The rules of this section apply to every task and every program. Section 3.1 gives the exceptions for untracked tasks.

Rule N3.1 (the owner runs the task). A task runs at the Neutrino that owns its destination range. The PE of the descriptor must be a PE of that node. A join and a spawn have no PE. A task with a PE of another node causes the error `Invalid` when it enters the tables of the owner. An untracked task is the exception (Rule N3.9).

Rule N3.2 (writes stay in the destination range). A task must write only inside its destination range. A write into the memory of another node is therefore a task of that node: a DMA task on a DMA engine of that node, which pulls the bytes from the source. The other way is an untracked task of the writer (Section 3.1). No Neutrino issues a task to a PE of another node, and no message reports the issue or the completion of a task to another node.

Rule N3.3 (forwarded invoke). An invoke whose destination range belongs to another Neutrino is a forwarded invoke. The requester puts the descriptor into its outbound invoke table and sends it to the owner as one Invoke message in the invoke channel. The `invoke` instruction returns when the descriptor is in the outbound invoke table. Its handle has version 0. From its arrival on, the task lives in the tables of the owner and runs on a PE of the owner (Rule N3.1).

Rule N3.4 (no forwarded spawn). The `done` range of a `spawn` must belong to the parent. A spawn whose `done` range belongs to another Neutrino causes the error `Unsupported`.

Rule N3.5 (remote dependencies). The owner of a task resolves the dependencies of the task when the task enters its tables. A dependency whose key the owner of the task owns is a local

lookup (Section 7.4). Every other dependency is a remote dependency: the owner of the task opens an entry in its outbound dep table and sends Await to the owner of the key. A remote dependency counts as one in the depcount of the task until AwaitAck(done) or Resolve arrives. A `wait` on a key of another Neutrino uses the same protocol.

Note. For a forwarded invoke, the owner of the task is the requester of each remote dependency, also for a key that the Neutrino that forwarded the invoke owns. Figure 7 shows this case.

Rule N3.6 (the owner assigns the version). The owner gives a versioned task the next version of its base address when the task enters the tables of the owner (Section 7.3). A forwarded invoke carries no version.

Rule N3.7 (version inference by the requester). A program that forwards a versioned invoke must name the version of that write by inference: the requester counts its own invokes on the base address since the last reset of the counter at the owner. The inference is exact only when both conditions are true:

1. No other Neutrino invokes a write of that base address after the last reset, or the program orders those invokes before its own invoke with `wait` instructions.
2. The program knows the value of the counter after the last reset: 0 after a program start at the owner, after `vreset_all` or after `vreset`, or the value of a `version` instruction of the program of the owner.

Note. A wrong inference makes a consumer name the wrong version: the consumer waits for a version that no task produces, or it does not wait for the write that it reads. Document 3 gives the check of the inference in r2-sim (Section 4.4).

Rule N3.8 (program order per requester). The owner enters the forwarded invokes of one requester into its tables in the program order of the requester. This is true with ready/valid flow and with ticket flow (Section 9.8). There is no order between the forwarded invokes of two requesters.

Example. In Listing 11, the program of `root.gpc0.sm0` copies its SMEM into the SMEM of `root.gpc0.sm1`. The destination range is in the SMEM of `sm1`, so the descriptor names the engine `root.gpc0.sm1.dma` and the invoke is forwarded to `sm1`. The engine of `sm1` pulls the bytes from the SMEM of `sm0`. The program of `sm0` infers the versions 1 and 2 of each buffer and waits for them with remote waits.

3.1 Untracked tasks

An **untracked task** is a task whose descriptor has the flag `untracked` (Table 2). It is for a write that no task depends on, for example the store of an output tile from the SMEM of an SM into the memory of the device. An untracked task differs from the other tasks, the **tracked tasks**, in two points:

- It runs at the Neutrino that invokes it, and it can write the memory of any node.
- No lookup finds it. No task can depend on it, and no `wait` can name it. Only a fence and the return of the program wait for it.

Rule N3.9 (the invoker runs an untracked task). An untracked task runs at the Neutrino that executes its invoke. The PE of the descriptor must be a PE of that node. The destination

range can be in any memory that the PE reaches, also in the memory of another node. The task must write only inside its destination range. A join and a spawn cannot be untracked.

Rule N3.10 (no key for an untracked task). An untracked task has no version, and no lookup finds it. The flag `versioned` of its descriptor must be false. Its invoke returns no handle. No dependency and no `wait` can name the task. Only a fence (Rule N3.11) and the return of the program (Rule N8.1) order later work after it.

Rule N3.11 (fence). The instruction `wait` with the key 0 is a **fence**. It blocks the program until every untracked task of the program is done. The key 0 names no producer, because no node owns address 0 (Rule N2.1). A descriptor must not list it in `deps`.

The dependencies of an untracked task are the same as those of a tracked task. The task waits in the scoreboard of the invoking Neutrino until its keys are done, and then it issues to its PE. A remote key uses the dependency protocol (Section 9.3). The untracked task sends no message of its own: its invoke is never forwarded.

The return of a program also waits for its untracked tasks (Rule N8.1). So when a program starts on a Neutrino, no untracked task of an earlier program is in flight there.

The owner of the destination range does not see an untracked task. The program orders the hazards of the range:

- A task that reads the range, or writes it again, comes after a fence (read-after-write and write-after-write).
- The untracked task comes after the earlier readers and the earlier writer of the range: it depends on their outputs, or the program waits for them before the invoke (Rule N7.1). After an earlier untracked writer, this wait is a fence.
- A `load` of the range comes after a fence. No table records the write of an untracked task, so no check of Rule N5.2 sees it.
- The parent of the program can read the range after the program returns: the return waits for the untracked task (Rule N8.1).

Example. In Listing 1, SM `root.gpc0.sm0` stores 64 KiB of its SMEM into the DRAM of its cluster, `root.gpc0.hbm`, as four untracked DMA tasks on its own engine. The cluster owns that DRAM, but its Neutrino takes no part: the SM sends no message for the stores. The fence at cycle 256 blocks the program until the four stores are done. Then the program signals the device with a join, a forwarded invoke into the sync-only space of the device, and the device program waits for that signal. Without the signal, the return of the SM program alone orders its parent after the stores.

Listing 1: Untracked stores, a fence and a signal: the program of SM `root.gpc0.sm0`.

```
.program root.gpc0.sm0 micro_sm    ; parent root.gpc0, started at cycle 164
    dma untracked dst=root.gpc0.hbm+0x0:0x4000 pe=root.gpc0.sm0.dma deps=[] from=root.gpc0.sm0.smem+0x0 to
     ↪ =root.gpc0.hbm+0x0 bytes=16384 tile=16384 label=0    ; t=164
    dma untracked dst=root.gpc0.hbm+0x4000:0x4000 pe=root.gpc0.sm0.dma deps=[] from=root.gpc0.sm0.smem+0
     ↪ x4000 to=root.gpc0.hbm+0x4000 bytes=16384 tile=16384 label=1    ; t=187
    dma untracked dst=root.gpc0.hbm+0x8000:0x4000 pe=root.gpc0.sm0.dma deps=[] from=root.gpc0.sm0.smem+0
     ↪ x8000 to=root.gpc0.hbm+0x8000 bytes=16384 tile=16384 label=2    ; t=210
    dma untracked dst=root.gpc0.hbm+0xc000:0x4000 pe=root.gpc0.sm0.dma deps=[] from=root.gpc0.sm0.smem+0
     ↪ xc000 to=root.gpc0.hbm+0xc000 bytes=16384 tile=16384 label=3    ; t=233
    wait 0    ; t=256
    join dst=root.sync+0x80000:0x200 deps=[] label=4    ; t=823
```

4 Task descriptor

A **descriptor** is the data structure of one task. The program builds it in the scratch memory of its Neutrino and submits it with `invoke` (Section 5). For a forwarded invoke, the Invoke message carries it to the owner. The PE reads the request data of the descriptor from the scratch memory of its Neutrino when it runs the task.

4.1 Fields

Table 2: The fields of a descriptor. The last column gives the bytes of the field in an Invoke message.

Field	Content	Bytes
`opcode`	The operation (Table 3). It selects the kind of PE and the form of `args`.	2
`pe`	The PE that runs the task. It must be a PE of the owner of `dst`, or of the invoking node for an untracked task. None for `join` and `spawn`.	2
`dst`	The destination range: `base` (8 B), `bound` (4 B) and `mask` (4 B), in granules (Section 2.3). It is the handle of a tracked task: later lookups find the task by it (Section 7.4).	16
`versioned`	True: `dst` is a versioned data range (Section 7). False: `dst` is unversioned, for example a signal in the sync-only space.	–
`untracked`	True: an untracked task (Section 3.1). It runs on a PE of the invoking node, and no lookup finds it. Its invoke is never forwarded, so no message carries the flag.	–
`deps`	Zero or more keys (Section 7.2). The task is ready when the producers of all keys are done.	8 each
`args`	The request data of the opcode: addresses, sizes, shapes and types, never data values (Table 3).	per opcode
`label`	An identity that software assigns, for example a layer and a step. It has no hardware function. Traces name tasks by it (document 4).	–

A dependency is one 64-bit word: a granule address and a version. In the 2p configuration a byte address has 38 bits and the granule is 512 B, so a granule address has 29 bits and 35 bits are free for the version. The word 0 is the key of the fence: granule 0 holds the null address, which no node owns (Rules N2.1 and N3.11). The Invoke message does not carry `label`.

The list `deps` gives the edges of the task graph. It is not the read set of the task.

Rule N4.1 (dependencies are edges). A descriptor must list the key of each producer whose output the task reads. Two cases need no key: the program waits for the key before the invoke, or no task of the program produces the data. Weights in memory before the program starts are an example of the second case.

Table 3: The opcodes. The mnemonic is the name in listings (Section 5.8). The last column gives the bytes of `args` in an Invoke message.

Id	Name	Mnemonic	PE	Arguments	Bytes
0	`join`	`join`	none	none	0
1	`spawn`	`spawn`	none	`child`	4
2	`dma`	`dma`	DMA engine	`src`, `dst`, `bytes`, `tile_bytes`	32
3	`atlas.matmul`	`mm`	Atlas	`a`, `b`, `a2`, `b2`, `lda`, `ldb`, `ldc`, `m`, `n`, `k`, three types, `accumulate`, `keep`, `tile`	52, +8 for each of `a2`, `b2`
4	`muon.vector`	`vec`	Muon	`ops`, `bytes_in`, `bytes_out`, `src`	24, 32 with `src`

4.2 Opcodes

Opcodes 0 and 1 have no PE. The opcodes from 2 up are PE operations. More PE operations can be added. Document 3 gives how r2-sim numbers them and names their mnemonics (Section 12.1).

Rule N4.2 (opcode and PE). The PE of a descriptor must be a PE of the kind that the opcode names: a DMA engine for `dma`, an Atlas for `atlas.matmul`, a Muon for `muon.vector`.

4.2.1 `dma`

A DMA task copies `bytes` bytes from `src` to `dst`.

Argument	Meaning
`src`	The source address. It can be in the memory of any node.
`dst`	The destination address. The bytes [`dst`, `dst` + `bytes`) must lie in the destination range of the task (Rule N3.2).
`bytes`	The number of bytes.
`tile_bytes`	The largest burst that the program asks for (document 2, Section 5.1). It must be greater than 0. The burst size of the engine, when the engine has one, replaces it.

4.2.2 `atlas.matmul`

A matmul task computes $C = AB$ or $C = C + AB$ on an Atlas, with A of $m \times k$, B of $k \times n$ and C of $m \times n$ elements, all row major. C is at the base of the destination range.

Argument	Meaning
`a`, `b`	The addresses of A and B. With `a2`: the low half rows of A. With `b2`: the low half columns of B.
`a2`, `b2`	The addresses of the high half rows of A and of the high half columns of B, or none.
`lda`, `ldb`, `ldc`	The leading dimensions of A, B and C, in elements.
`m`, `n`, `k`	The shape.
`in_dtype`, `b_dtype`, `out_dtype`	The element types of A, B and C: `bf16`, `f32`, `fp8` or `mxfp4`.
`accumulate`	True: $C = C + AB$.
`keep`	True: C stays in the accumulator of the Atlas for the next accumulate task on the same destination range. Without an accumulator it has no effect.
`tile`	The output tile [rows, columns]. The Atlas computes the task one output tile at a time (document 2, Section 5.2).

The elements of C that the task writes, $((m-1) \cdot \texttt{ldc} + n)$ elements from the base, must lie in the destination range (Rule N3.2).

Note. Split operands put the rows of A in two half matrices, for a banked SMEM (document 2, Section 3.1). Row r of A, with $t = \lfloor r/tile_0 \rfloor$, $j = r \bmod tile_0$ and $h = tile_0/2$, is row $th + j$ of the matrix at `a` when $j < h$, else row $th + j - h$ of the matrix at `a2`. The columns of B split over `b` and `b2` in the same way with $tile_1$.

4.2.3 `muon.vector`

A vector task runs `ops` lane operations on a Muon. The descriptor gives the work and the bytes of the task. The code of the Muon is outside this document.

Argument	Meaning
`ops`	The lane operations of the task.
`bytes_in`	The bytes that the task reads, from the memory block that holds `src`.
`bytes_out`	The bytes that the task writes, to the memory block that holds the base of the destination range.
`src`	The address that the task reads. None: the block of the destination range.

4.2.4 `join`

A join task has no PE and no arguments. It is ready when its dependencies are done, and it completes after the completion cost `c_complete`. Its destination range must be in the sync-only space (Rule N2.5): a join writes no data.

4.2.5 `spawn`

A spawn task has no PE. Its argument `child` names the child Neutrino (4 bytes). Only the `spawn` instruction makes a spawn task (Section 5.3). The destination range of a spawn task is the `done` range of the instruction, in the sync-only space of the parent (Rule N2.5); the task is unversioned.

4.3 Size in a message

The bytes of a descriptor in an Invoke message are

$$2 + 2 + 16 + 8\,|\texttt{deps}| + |\texttt{args}|,$$

with |`args`| from Table 3. A message is $\lceil \textit{bytes}/\texttt{msg_bytes} \rceil$ beats of `msg_bytes` (64 B by default), and at most 4 beats.

Rule N4.3 (size of a forwarded descriptor). A forwarded descriptor must fit in 4 beats: 256 B with `msg_bytes` 64. A larger one causes the error `Invalid`. A local invoke has no such limit. The limit of 4 beats is a constraint of the implementation, not of the architecture.

Table 4: Sizes of forwarded descriptors with `msg_bytes` 64.

Descriptor	Bytes	Beats	Most deps in 4 beats
`dma`, no deps	52	1	25
`mm`, no split operand, no deps	72	2	23
`mm`, no split operand, 1 dep	80	2	
`vec` with `src`, no deps	52	1	25
`join`, 20 deps	180	3	29

5 Instruction set

5.1 Programs and instructions

A program is a sequence of instructions to its Neutrino. The program has at most one instruction in progress. For each instruction the Neutrino does these steps:

1. It charges the cost of the instruction on its scalar resource (Section 5.7).
2. At the end of the cost, it executes the instruction.
3. It answers the program. The program then continues.

An instruction can stall (a table is full) or block (a `wait`). Then the answer comes later. Scalar work of the program between two instructions is charged with the instruction `cycles`.

The in-memory form of a program is its definition: the sequence of instructions and the descriptors, as the data structures that the program passes to its Neutrino. The assembly text of Section 5.8 is an experimental byproduct: software prints it from the recorded instructions. There is no assembler: no tool reads the text back into a program. Document 3 gives the program interface of r2-sim (Section 4.2).

5.2 `invoke`

Operand: a descriptor (Section 4). Result: a handle, the base address of `dst` and a version, or no handle for an untracked task.

The Neutrino does these steps:

1. If the task is untracked, steps 2 and 3 do not apply. The task enters the local tables as in step 4, without a version, and the invoke returns no handle (Section 3.1).

Table 5: The instructions. The costs are in Table 6.

Instruction	Effect	Cost
`invoke`	Submits a task. Returns the handle (base of `dst`, version). An untracked invoke returns no handle.	c_{desc} + `c_invoke`
`spawn`	Starts a program on a direct child when `deps` are done; the task `done` completes at the return of the program.	c_{desc} + `c_spawn`
`wait`	Blocks until the producer of the key is done. With the key 0 (a fence): until every untracked task of the program is done.	`c_wait`
`version`	Sets the version counter of base address `a` to `v`.	`c_version`
`vreset`	Frees the version table entry of base address `a`, at the owner.	`c_vreset`
`vreset_all`	Clears the version table of the Neutrino.	`c_version`
`cycles`	Charges `n` cycles of scalar work.	`n`
`load`	Reads `len` bytes from memory.	`c_load` + memory
`store`	Writes bytes to memory.	`c_store` + memory

2. It finds the owner of `dst` (Rule N2.2).
3. If the owner is another Neutrino, the invoke is forwarded (Rule N3.3). When the outbound invoke table is full, the instruction stalls until an entry frees. The handle has version 0.
4. Else the task enters the local tables (admission, Section 6.2). The instruction stalls until an entry frees in three cases: the scoreboard is full, the issue headroom is used up, or the outbound dep table has fewer free entries than the task has remote dependencies.
5. The handle of a local versioned invoke has the version that the Neutrino assigns. The handle of a local unversioned invoke has version 0.
6. When the task has no dependency that is not done, it is ready at once.

The errors of a local invoke are in Table 13: a PE of another node, an empty destination range, a full pending table, a full version table, a dependency on the task itself, a version that a task in the tables still holds, and more remote dependencies than the outbound dep table can hold.

5.3 `spawn`

Operands: a direct child Neutrino, a range `done` of the parent, a list of keys `deps`, and the program to start. Result: the handle (base of `done`, version 0).

1. The child must be a direct child of the Neutrino. Else this is the error `Invalid`.
2. The Neutrino makes a spawn task: opcode `spawn`, no PE, unversioned destination range `done`, dependencies `deps`. The task enters the tables like a local invoke. `done` must be in the sync-only space of the parent (Rules N2.5 and N3.4).
3. When the task is ready, it enters the execution table and the Neutrino sends Start with the program to the child (Section 9.7).
4. The task completes when Return arrives from the child. A `wait` on `done` therefore waits for the child program. A join can depend on `done` like on any other key.

5.4 `wait`

Operand: a key (Section 7.2): (a, v) for a versioned range, an address a for an unversioned range, or 0 for a fence. The instruction blocks until the producer of the key is done.

- When the Neutrino owns a, it looks the key up in its tables (Section 7.4). A done key answers at once. Otherwise the program waits for every entry that the lookup returns; it continues

`c_resolve` cycles after the last one completes. A key with no producer yet makes a pending entry. A full pending table is the error `PendingFull`.

- When another Neutrino owns a, the Neutrino opens an entry in its outbound dep table and sends Await (Section 9.3). It answers at AwaitAck(done) or at Resolve. When the outbound dep table is full, the instruction stalls until an entry frees.

An unversioned wait on a waits for the newest unversioned producer whose `dst` holds a, and for every other producer that holds a and is not done (Section 7.4).

A `wait` with the key 0 is a fence (Rule N3.11). It does no lookup. It answers at once when no untracked task is in the tables of the Neutrino. Otherwise the program continues `c_resolve` cycles after the last of these tasks completes.

5.5 `version`, `vreset` and `vreset_all`

`version a, v` sets the counter of the base address `a` in the version table of the Neutrino to `v`. It makes the entry when it does not exist. The next local invoke of a range with base `a` gets version `v` + 1. The Neutrino must own `a`: `version` of the range of another Neutrino is the error `Invalid`. A full version table is the error `VersionsFull`.

Rule N5.1 (wait before version). Before a `version` instruction sets the counter of a base address, the program must wait for the last version of that base that it expects.

Note. Without the wait, an invoke that produces an expected version can still be on its way, for example a forwarded invoke. The local instruction stream is in order, but such an invoke can arrive after the `version` instruction. It then gets a later version, and a consumer that waits for the expected version never wakes. The Neutrino does not release such a waiter: a released waiter would continue without its data. The order is the job of the program.

`vreset a` frees the version table entry of the base address `a` at its owner. An absent entry is not an error. The instruction routes by address:

- When the Neutrino owns `a`, it frees the entry at once.
- Else it sends VersionReset to the owner in the invoke channel, behind the earlier forwarded invokes of the program to that owner (Section 9.6). The instruction stalls while the outbound invoke table is full.

`vreset_all` clears the version table of the Neutrino. The start of a program on a Neutrino does the same (Section 8.5).

A producer that is still in the scoreboard or the execution table keeps its version after a reset: lookups of its key find the task entry. After the task retires, a lookup of a version from before the reset finds no counter (Section 7.7).

5.6 `cycles`, `load` and `store`

`cycles n` charges `n` cycles on the scalar resource. It stands for the scalar work of the program, for example the plan of a window of tasks.

`load a, len` reads `len` bytes at `a`. The Neutrino is a master on the data buses of its node. The request takes the path to the memory block of `a` and the latency of the block; the data returns over the path at the bandwidths of its blocks. The value is the memory state after the last writer of the range. A load is the only place where a program reads data: data-dependent control flow uses it.

`store a, bytes` writes the bytes at `a`. The data takes the path to the memory block and its write port. The program continues when the data is written: after the latency of the path and the time of the bytes on its blocks. The memory block adds no latency to a write (document 2, Section 5.1).

A load or a store is split at the boundaries of memory blocks and at the transaction size of each block on its path (document 2, Section 2.3).

Rule N5.2 (no load during a write). A program must not load a range while a task that writes the range is issued and not done. Such a load is the error `Race`. No table records the write of an untracked task: a fence before the load is the job of the program (Section 3.1).

5.7 Costs

Every cost is a parameter of the Neutrino (Section 11). The Neutrino charges the costs of its instructions on its **scalar resource**, at the rate `scalar_rate` (1 per cycle by default), so the costs of successive instructions add. It charges `c_complete` on a separate resource, the completion port, at the rate `complete_rate`: completions and instructions do not delay each other.

The cost of the descriptor assembly, before `invoke` and `spawn`, is

$$c_{\text{desc}} = \texttt{c_base} + \texttt{c_dep} \cdot |\texttt{deps}| + \texttt{c_word} \cdot \lceil \mathit{bytes}/8 \rceil,$$

with *bytes* the size of the descriptor in a message (Section 4.3). For a spawn, $\mathit{bytes} = 24 + 8\,|\texttt{deps}|$.

Table 6: The cost parameters of a Neutrino and their defaults.

Parameter	Default	Charged for
`c_invoke`	8	The issue of an assembled descriptor (`invoke`).
`c_base`	8	Descriptor assembly: a fixed part.
`c_dep`	3	Descriptor assembly: per dependency.
`c_word`	1	Descriptor assembly: per 8-byte word of the message.
`c_spawn`	16	`spawn`, after its descriptor assembly.
`c_wait`	2	`wait`, before the resolution time.
`c_version`	1	`version` and `vreset_all`.
`c_vreset`	1	`vreset`, plus the message when remote.
`c_load`	1	The issue of a `load`; the memory adds its time.
`c_store`	1	The issue of a `store`; the memory adds its time.
`c_complete`	4	Each completion, on the completion port.
`c_resolve`	0	A delay, on no resource. It starts when the depcount of a task becomes 0 after its invoke, and ends when the task is ready. It also delays the wake of a program after the key of its local wait is done.
`c_plan`	0	The scalar work of a driver per planned task, as `cycles`.

Example. With the defaults:

- a join with no dependency (20 B, 3 words): $c_{\text{desc}} = 8 + 3 = 11$, and the `invoke` costs $11 + 8 = 19$ cycles;
- a join with one dependency (28 B, 4 words): $c_{\text{desc}} = 8 + 3 + 4 = 15$, and the `invoke` costs 23 cycles;

- a `dma` with no dependency (52 B, 7 words): $c_{\text{desc}} = 15$, and the `invoke` costs 23 cycles;
- a `spawn` with no dependency (24 B, 3 words): $c_{\text{desc}} = 11$, and the `spawn` costs $11 + 16 = 27$ cycles.

The cycles in Listings 6, 9 and 11 show these costs.

5.8 Assembly text

The assembly text is an experimental feature. Software prints one listing per program instance. A listing has one line per instruction, in program order. The descriptor of a task is the operand list of its line. Document 3 gives the files that r2-sim writes (Section 11.6).

The first line is a header:

```
.program <neutrino> <name>     ; parent <neutrino>, started at cycle <t>
.program <neutrino> <name>     ; started by the host at cycle <t>
```

Each instruction line ends with the comment `; t=<cycle>`: the cycle at which the program made the request, before its cost. The character `;` starts a comment.

Table 7: The syntax of the instructions in a listing. Brackets mark optional operands.

Instruction	Syntax
task	`<mnemonic> [untracked] dst=<range>[@v<n>] [pe=<path>] deps=[<key>,...] <args> [label=<n>]`
`dma` arguments	`from=<address> to=<address> bytes=<n> tile=<n>`
`mm` arguments	`a=<address> b=<address> [a2=<address>] [b2=<address>] lda=<n> ldb=<n> ldc=<n> m=<n> n=<n> k=<n> types=<a>,<b>,<c> acc=0|1 keep=0|1 tile=<rows>x<cols>`
`vec` arguments	`ops=<n> in=<bytes> out=<bytes> [src=<address>]`
`spawn`	`spawn child=<neutrino> done=<range> deps=[<key>,...]`
`wait`	`wait <key>`; the fence: `wait 0`
`version`	`version <address> v=<n>`
`vreset`, `vreset_all`	`vreset <address>`, `vreset_all`
`cycles`	`cycles <n>`
`load`, `store`	`load <address> len=<n>`, `store <address> len=<n>`
`mark`, `cache_flush` (r2-sim only)	`mark "<name>"`, `cache_flush`

The values have these forms:

- A number is decimal. An offset or a length is hexadecimal with `0x`.
- An address is `<symbol>+0x<offset>`: relative to the narrowest named range that holds it. The named ranges are the memories (a memory behind a hash link at its linear range), the banks outside the range of their memory, the linear ranges of hash links that no memory covers, and the sync-only ranges. An address in no named range is a bare `0x` number.
- A range is `<address>:0x<length>` in bytes, followed by `/0x<mask>` when its mask is not all ones. The mask counts granules (Section 2.3).
- A versioned key is `<address>@v<n>`. An unversioned key is the address alone.
- After the range of a versioned task, `@v<n>` is the version that the program infers. `@v?` means that the program gave no inference, for example for a local invoke, whose handle carries the version.
- PEs and Neutrinos are dotted paths: `root.gpc0.sm0.dma`.

Example. This line is a DMA task on the engine of `root.gpc0.sm1`. It writes version 2 of the 16 KiB range at offset `0x80000` of the SMEM of that SM, from the SMEM of `root.gpc0.sm0`:

```
dma dst=root.gpc0.sm1.smem+0x80000:0x4000@v2 pe=root.gpc0.sm1.dma deps=[] from=root.
    ↪ gpc0.sm0.smem+0x0 to=root.gpc0.sm1.smem+0x80000 bytes=16384 tile=16384
```

6 Tables

Figure 3 shows a Neutrino with its tables, its program interface, its PE ports and its message channels. The local tables hold the tasks that the Neutrino owns and the versions of its ranges. The outbound tables hold the requests that the Neutrino sends as a **requester**. The inbound tables hold the requests that it receives as an owner.

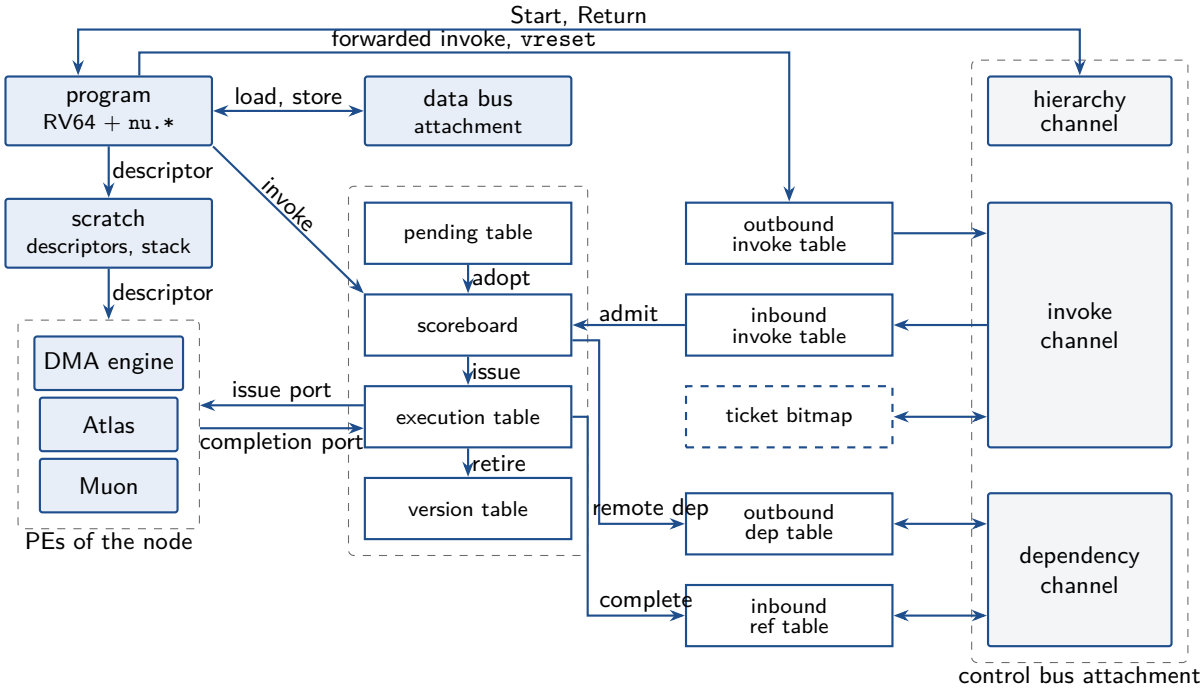


Figure 3: A Neutrino. The program submits tasks to the local tables or, for a remote destination range, to the outbound invoke table. The scoreboard issues ready tasks to the PEs, and the completion port returns them. A remote dependency or a remote `wait` opens an entry in the outbound dep table. The inbound tables take the requests of other Neutrinos from the control bus. The ticket bitmap exists in ticket flow only.

6.1 Task entries and tags

A **task entry** is the record of one task from its invoke to its retire. Its **tag**, the task instance ID on the issue port, does not change. The scoreboard and the execution table are two capacity counts over the same task entries. A task entry is in one of three phases:

- scoreboard: invoked, with a depcount;
- executing: issued to a PE, or a task without a PE in progress;
- done: completed and unversioned. A versioned task retires at its completion and has no done phase.

Table 8: The tables of a Neutrino. "Key" is the configuration key of the capacity; Table 15 gives the values.

Table	Side	Purpose	An entry	Key
pending table	local	Keys that a lookup names before the invoke of their producer.	The key; its waiters.	`pending`
scoreboard	local	Invoked tasks that are not issued.	A task entry: descriptor, depcount.	`scoreboard`
execution table	local	Issued tasks: executing, then done until retire.	A task entry: `dst`, version, state, waiters.	`execution`
version table	local	Versioned base addresses.	The base address; the version counter.	`versions`
inbound ref table	owner	Remote requests registered on keys that are not done.	The requester; its request; the entries to wait for.	`inbound_refs`
inbound invoke table	owner	Forwarded invokes that arrived and wait to enter the scoreboard.	One Invoke message.	`inbound_invokes`
outbound dep table	requester	Open remote dependencies and remote waits.	The owner; the key; the purpose; the state.	`outbound_deps`
outbound invoke table	requester	Forwarded invokes and VersionReset messages that have not left (ready/valid) or are not acknowledged (ticket).	One message.	`outbound_invokes`
ticket bitmap	owner	Requesters that hold a ticket (ticket flow only).	One bit per other Neutrino.	none

A pending entry has a tag of its own.

An untracked task (Section 3.1) has a task entry in the tables of the invoking Neutrino, with a tag and a depcount, and no version. A lookup does not find the entry. The task has no done phase: it retires at its completion. The Neutrino counts its untracked tasks that are not retired. A fence and the return of the program wait until the count is 0.

At invoke, the Neutrino looks up each dependency once and resolves it to tags (Section 7.4). The scoreboard entry counts the tags that are not done: its **depcount**. A completion broadcasts its tag. Each scoreboard entry that holds the tag decrements its depcount, as an entry of an issue queue does. Program waits and inbound refs on the tag are satisfied in the same way. A lookup is a range compare over the entries of the tables. A completion is a compare of one tag. No step needs a comparator array of all entries against all entries.

6.2 Scoreboard and admission

The scoreboard holds the invoked tasks that are not issued. Only scoreboard entries have a depcount. A task issues when its depcount is 0 and its PE can take it (Section 8.2).

A task enters the scoreboard (**admission**) from a local invoke or from the head of the inbound invoke table. Admission needs every resource of the task at once:

- a free scoreboard entry;
- issue headroom (Rule N6.1);

- for a new versioned base address, a free version table entry (an untracked task needs none);
- a free pending entry for each key that no producer has yet;
- a free outbound dep entry for each remote dependency.

Rule N6.1 (issue headroom). A task enters the scoreboard only while

$$\mathit{scoreboard} + \mathit{execution} < \texttt{execution} - \texttt{issue_headroom},$$

with *scoreboard* and *execution* the entries in use and `execution` the capacity of the execution table. The default of `issue_headroom` is the number of PEs of the node. The configuration must give `execution` $>$ `issue_headroom` and `scoreboard` ≥ 1.

Note. The headroom keeps execution table entries for tasks that are ready to issue. Without it the scoreboard can fill again each time an execution entry frees, and execution entries free only through completions that need execution entries: a deadlock. With the rule, the sum of the two tables never exceeds `execution` $-$ `issue_headroom`, so an issue always finds a free execution entry.

6.3 Execution table

The execution table holds the issued tasks: executing, then done. A versioned task retires at its completion, because the version table carries its state from then on. An untracked task also retires at its completion, because no lookup can find it. An unversioned tracked task stays done until one of these events:

- a new unversioned producer whose `dst` holds every granule of its `dst` is invoked (the new task supersedes it);
- the program of the Neutrino returns.

A consumer that the program invokes after the completion therefore still finds the done producer. Software recycles signal ranges.

If the execution table is full when a task can issue, the task waits for a free entry. The issue headroom keeps entries free for issue, so the wait shows first at admission. The backpressure then goes back step by step: the scoreboard takes no more tasks, the head of the inbound invoke table waits, the invoke channel fills, and the programs that invoke into this Neutrino stall (Section 8.3).

6.4 Pending table

A lookup that finds no producer for its key makes a **pending entry**: a dependency of an invoke, a `wait`, or an Await from another Neutrino. The entry holds the key and its waiters: local scoreboard entries, local program waits and inbound refs. The invoke of the producer of the key adopts the entry (Section 7.5): the waiters move to the task entry, and the pending entry frees.

6.5 Version table

The version table has one entry per versioned base address: the base address and its version counter. There are no other records: an entry does not hold the tags of the versions that are not done. The first versioned invoke of a base address makes its entry, and so does `version`. `vreset`, VersionReset, `vreset_all` and the start of a program free entries.

Rule N6.2 (live versioned ranges). Software must keep the number of live versioned base addresses of each owner within the capacity of its version table. A full version table is the error `VersionsFull` for a local invoke or a `version`. A forwarded invoke that needs a new entry waits at the head of the inbound invoke table until an entry frees.

Note. An entry is about 56 bits in the 2p configuration: a base of 24 bits (a granule address in an own space of 33 bits) and a counter of 32 bits. That is 1.75 KiB for 256 entries and 7 KiB for 1024 entries. The memory owners (the clusters in the 2p configuration) hold one entry per output block in flight, so their tables are larger (1024) than those of the SMs (256). An entry holds no bound, so the Neutrino does not check Rule N7.3.

6.6 Inbound ref table

The inbound ref table holds the remote requests that the owner registered: an Await whose key is not done. An entry holds the requester, the request ID of the requester, and the number of entries that it waits for (more than one for an unversioned key with several unordered producers). When the last of them completes, the owner sends Resolve and the entry frees. A full inbound ref table rejects new Await messages (Section 9.3). Remote readers are waiters only: the owner counts nothing for them.

6.7 Inbound invoke table

The inbound invoke table is the last stage of the invoke channel at the owner (Section 9.2). It holds forwarded invokes and VersionReset messages in arrival order. Its capacity counts messages: a message of four beats is one entry. Under ready/valid flow, an Invoke reserves an entry when it leaves the last stage of the channel, so the table never overflows.

The head of the table enters the scoreboard when all of its resources are free (Section 6.2). Otherwise the head stays and the entries behind it wait: the table is first in, first out. The owner retries the head when a scoreboard entry, a pending entry, a version table entry, an outbound dep entry or issue headroom frees.

6.8 Outbound dep table

The outbound dep table holds the remote dependencies of the tasks of the Neutrino and its remote program waits. An entry holds the owner of the key, the key, the purpose (a dependency of a task, or a program wait), the state (sent or registered) and the number of rejects. The entry frees at AwaitAck(done) or at Resolve.

Rule N6.3 (remote dependencies of one task). A task must not have more remote dependencies than the capacity of the outbound dep table of its owner. Such a task causes the error `Invalid`.

6.9 Outbound invoke table

The outbound invoke table is the first stage of the invoke channel at the requester. It holds forwarded invokes and VersionReset messages in program order. Under ready/valid flow an entry frees when its message leaves the table. Under ticket flow an entry frees when its InvokeAck arrives (Section 9.5). The program stalls on `invoke` or `vreset` while the table is full.

6.10 Ticket bitmap

In ticket flow, each owner has a ticket bitmap: one bit for each other Neutrino, $N - 1$ bits for N Neutrinos. A set bit means that the requester holds a ticket and that the owner rejects its requests without a ticket. The owner also keeps the round-robin pointer of its grants and the requesters with a grant in flight. A requester keeps at most one ticket per owner, and the requests to replay for it.

6.11 Full tables

A table without a free entry stops the program, holds the request of another Neutrino, or causes an error. Table 9 gives the case of each table and each request.

Table 9: What happens when a table has no free entry. "Stall": the program waits and retries when an entry frees. "Waits": the forwarded invoke stays at the head of the inbound invoke table. "Reject": the owner answers AwaitAck(reject). "Error": an error (Section 8.4).

Table	`invoke`	Forwarded invoke at the owner	`wait`	Await at the owner
scoreboard, issue headroom	stall	waits	–	–
pending table	error	waits	error	reject
version table	error	waits	–	–
inbound ref table	–	–	–	reject
outbound dep table	stall	waits	stall (remote key)	–
outbound invoke table	stall (remote `dst`)	–	–	–
inbound invoke table	–	ready/valid: the message waits in the last stage; ticket flow: RetryAck	–	–
execution table	the ready task waits for an entry	same	–	–

7 Dependencies and versions

7.1 Hazards

The Neutrino tracks read-after-write order only: a task waits for the producers of the keys that it names. Every task writes a range, so every order is a read-after-write order on some range. The hardware counts no readers. Write-after-read and write-after-write order are the job of software, for local and remote ranges alike.

Rule N7.1 (writers after readers). A task that writes a range must come after the tasks that read the earlier value of the range. It must depend on their outputs, or the program must wait for their outputs before the invoke. A reader that produces nothing must produce a signal (a join) for this purpose. A task that writes a range again must also come after the earlier writer of the range. A dependency on the earlier version does this, or the readers that depend on it.

Note. An owner can count its local readers but not the readers on other nodes. A rule that changes with the place of the reader is a trap for a compiler, so the rule is the same everywhere.

Example. In Listing 12 the last `dma` writes version 31 of a weight buffer in the SMEM of `root.gpc0.sm1`, at offset `0xc0000`. It has two keys:

- `root.gpc0.sm1.smem+0x4d600@v4`: the output of the matmul that read version 30 of the buffer (write-after-read);
- `root.gpc0.sm1.smem+0xc0000@v30`: the earlier version of the buffer (write-after-write).

7.2 Keys

A **key** names a producer:

- A versioned key (a, v) names version $v \geq 1$ of the versioned range whose base is the granule a.
- An unversioned key a names the unversioned producers whose destination range holds the granule a (Section 2.3).

The **handle** that an invoke returns is the base of `dst` and a version. Its key is $(base, v)$ for $v > 0$ and the unversioned key $base$ for $v = 0$.

Rule N7.2 (a versioned key names a base). A versioned key must name the base of the destination range of its producer. A versioned key with another granule of the range finds no producer, and its consumer never becomes ready.

Rule N7.3 (one bound and one mask per base). While a base address has an entry in the version table, every versioned invoke of that base address must have the same bound and the same mask. The Neutrino does not check this rule: the version table holds no bound and no mask (Section 6.5).

Note. A versioned lookup compares base addresses only. The Neutrino does not compare versioned ranges with different bases, so it does not detect two live versioned ranges that hold a common granule.

7.3 Version assignment

A versioned task with base address a gets its version when it enters the tables of the owner: the local invoke, or the admission from the inbound invoke table. The version is the counter of a plus 1, or 1 when a has no counter. The counter takes the new value. The task entry carries the key (a, v) until it retires. The **version** therefore counts the writes of the range in the order in which the invokes enter the tables of the owner.

Rule N7.4 (a version in flight). An invoke must not get a version that a task entry in the tables still holds. This happens when a program resets a base address while its producer is in flight and then writes the range again. This is the error `VersionInFlight`.

7.4 Lookup

A **lookup** finds the state of a key in the tables of its owner. A dependency of an invoke, a local `wait` and an Await use the same lookup. A lookup does not see the entries of untracked tasks

(Rule N3.10), and a fence does no lookup. Table 10 gives the steps, in order. The first step that matches gives the result.

Table 10: The lookup of a key. "Wait": the dependency counts in the depcount until the named entries complete.

Step	Versioned key (a, v)	Unversioned key a
1	A pending entry with the key (a, v) exists: wait on it.	A pending entry with the key a exists: wait on it.
2	A task entry with the key (a, v) is in the scoreboard or the execution table: wait on it (in flight).	Unversioned task entries whose `dst` holds a exist: bind to the newest of them (in invoke order) and to every other one that is not done. Wait on the bound entries that are not done. When all bound entries are done, the key is done.
3	The counter of a is v or more: the key is done.	–
4	Else: make a new pending entry for (a, v) and wait on it. The invoke of the producer has not arrived.	No task entry holds a: make a new pending entry for a and wait on it.

The three outcomes of a lookup are:

- *done*: the dependency does not count;
- *in flight*: the dependency waits on task entries in the scoreboard or the execution table;
- *new*: the dependency waits on a pending entry for a producer whose invoke comes later. A consumer can therefore come before its producer, locally or from another Neutrino.

An older unversioned producer that is done does not bind when a newer one holds the address: it is an earlier value of a recycled range. Two or more bound entries that are executing at the same lookup are writers that the program did not order.

7.5 Adoption

An invoke adopts the pending entries of its keys:

- a versioned invoke with key (a, v) adopts the pending entry of (a, v);
- an unversioned invoke adopts every unversioned pending entry whose address is in its destination range.

The waiters of each adopted entry move to the task entry, and the pending entry frees. In hardware terms, adoption broadcasts the remap from the pending tag to the task tag.

Rule N7.5 (no dependency on itself). A task must not depend on its own key. For a versioned task, this is a dependency equal to its own key. For an unversioned task, this is an unversioned dependency in its own destination range that no other producer holds. Such a task causes the error `SelfDependency`.

7.6 Completion and retire

At the completion of a task, the Neutrino wakes the waiters of its entry:

- each scoreboard entry decrements its depcount; at 0 the task is ready `c_resolve` cycles later;
- each local program wait is satisfied when all its entries are done;

- each inbound ref decrements its count; at 0 the owner sends Resolve to the requester.

Then the entry retires or stays:

- A versioned task retires at once. Its counter in the version table says that its version is done.
- An untracked task retires at once. The count of the untracked tasks of the Neutrino falls by one. At 0, a fence or a return that waits continues.
- An unversioned task stays done. A later unversioned invoke whose `dst` holds every granule of its `dst` **supersedes** it: a done entry retires at that invoke, and an entry that is not done retires at its completion. The return of the program also retires the done unversioned entries, and the unversioned entries that are not done retire at their completion.

7.7 Version reset

These events free version table entries:

- `vreset a` of the owner, or a VersionReset message for `a` from another Neutrino: the entry of `a`;
- `vreset_all`: every entry of the Neutrino;
- the start of a program on the Neutrino: every entry (Section 8.5).

The return of a program does not free entries.

After a reset of a, the next invoke of a gets version 1. A task that is still in the scoreboard or the execution table keeps its key, and lookups find it. After it retires, the counter does not record its version: a lookup of a version from before the reset makes a pending entry that waits for a new producer.

Rule N7.6 (versions after a reset). After a reset of a base address, a program must not name a version of that base address from before the reset. The only exception is a version whose producer is still in the tables of the owner.

7.8 Version inference

A local versioned invoke returns its version in the handle. A forwarded invoke returns version 0, and the program infers the version (Rule N3.7). A program keeps one counter for each remote base address that it writes. It adds 1 to the counter for each forwarded invoke of the base address. It sets the counter to 0 when it sends a reset of the base address. Ordered delivery makes the count exact (Rule N3.8).

Example. In Listing 8 the device forwards four joins on the flag range `root.gpc0.sm0.sync+0x0`. The device infers the versions 1 to 4 and waits for `root.gpc0.sm0.sync+0x0@v4`. The SM assigns the same versions because the four invokes arrive in program order.

7.9 Joins and signals

A join is a task without a PE that writes no data. A consumer depends on a join, and the join depends on the producers. Uses:

- many producers to one consumer: the join collects up to 29 keys in a forwarded descriptor (Table 4);
- read-side order: the readers of a range produce joins, and the next writer depends on them (Rule N7.1);

- barriers: a join on the keys of all tasks of a phase.

Note. Versions can count arrivals. Each of x tasks writes a versioned range a with no dependency, on forwarded invokes. A wait on $(a, v_0 + x)$ then completes when the x-th write completes, with v_0 the counter before the first arrival. Each arrival costs a scoreboard entry, an execution entry, the invoke costs and one Invoke message.

8 Task lifecycle

Figure 4 shows the states of a task from its invoke to its retire. A local invoke enters the scoreboard of the Neutrino of the program. A forwarded invoke passes through the outbound invoke table of the requester, the invoke channel and the inbound invoke table of the owner first.

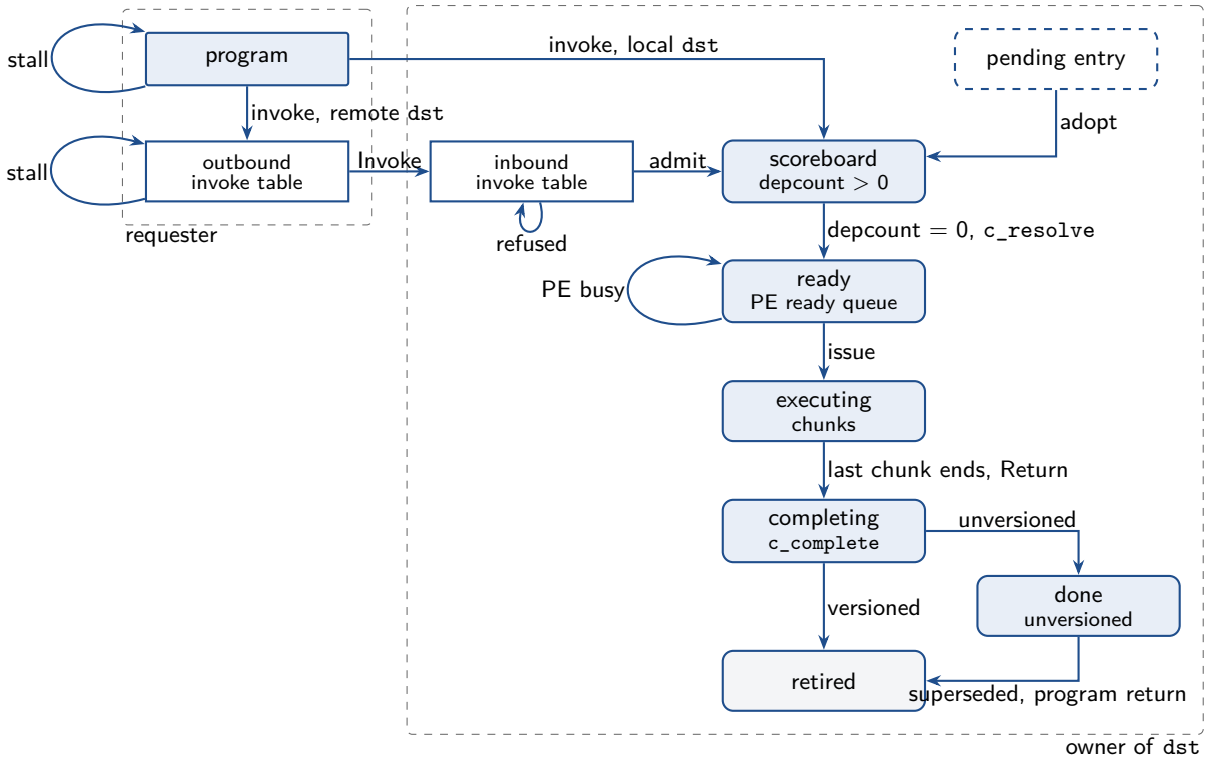


Figure 4: The task lifecycle. The loops are stalls of the program, refusals at the head of the inbound invoke table, and the wait of a ready task for its PE. A pending entry is a key that a lookup named before the invoke of its producer; the invoke adopts it.

8.1 From invoke to ready

1. The program builds the descriptor and executes `invoke`. The Neutrino charges c_{desc} + `c_invoke` on its scalar resource (Section 5.7).
2. For a local `dst` or an untracked task, admission puts the task into the scoreboard (Section 6.2). An untracked task gets no version. For a remote `dst`, the requester queues the descriptor; the owner admits the task from the head of its inbound invoke table later. In both cases the program continues after the instruction.
3. At admission the owner assigns the version (versioned `dst`), looks up each local key, adopts the pending entries that the new task produces (Section 7.5), supersedes older unversioned

Table 11: The states of a task.

State	Place	Entered when	Left when
queued	outbound invoke table	A forwarded invoke executes.	The message leaves the table. In ticket flow the entry stays until the InvokeAck.
in the channel	stages of the control hops	The message leaves the source queue.	The message enters the inbound invoke table.
inbound	inbound invoke table	The message arrives.	Admission.
scoreboard	scoreboard	Admission. The task gets its version.	The depcount is 0.
ready	ready queue of the PE	The depcount is 0, plus `c_resolve` when the last dependency resolves after the invoke.	Issue.
executing	execution table, PE	Issue.	The PE ends the task (a spawn: Return arrives).
completing	completion port	The PE reports the end.	`c_complete` cycles later.
done	execution table	Completion of an unversioned tracked task.	Superseded, or the program returns.
retired	none	Completion of a versioned task or of an untracked task, or the end of done.	–

entries, and sends an Await for each remote key. The depcount is the number of local entries that are not done plus the number of remote keys.

4. The depcount falls by one at each completion of a local entry that the task waits on, and at each AwaitAck(done) or Resolve of a remote key. At 0 the task is ready after `c_resolve` cycles. A task with a depcount of 0 at admission is ready at once.

8.2 Issue, execution and completion

A ready task with a PE enters the ready queue of its PE, which is first in, first out. The PE issues the head of its queue when all of these are true:

- the PE runs fewer tasks than its `inflight` limit (1 by default);
- the execution table of the owner has a free entry.

At issue the task moves from the scoreboard to the execution table, and the PE starts it. The issue order respects every dependency that the scoreboard resolved, so it is a valid sequential order of the tasks.

The PE runs the task in the time of its cost model (document 2, Section 5). At the end, the PE reports the end on the completion port. The Neutrino charges `c_complete` on the port. Then the tables complete the task (Section 7.6).

A task without a PE issues when it is ready. A join completes after `c_complete`. A spawn sends Start to its child and completes after `c_complete` when Return arrives.

8.3 Stalls and refusals

A **stall** stops the program until an entry frees. Then the Neutrino executes the instruction again. Table 12 lists the stalls. A blocked `wait` is not a stall: it ends when its key is done.

Table 12: The stalls of a program.

Stall	Cause	Instructions
`Scoreboard`	The scoreboard is full.	`invoke`, `spawn`
`Headroom`	The issue headroom is used up.	`invoke`, `spawn`
`OutboundDeps`	The outbound dep table has fewer free entries than the task has remote keys.	`invoke`, `spawn`, remote `wait`
`OutboundInvokes`	The outbound invoke table is full.	forwarded `invoke`, remote `vreset`

An owner never stalls on a request of another Neutrino. It refuses it:

- a forwarded invoke whose resources are not free stays at the head of the inbound invoke table (Section 6.7);
- an Await that needs an entry of a full inbound ref table or of a full pending table gets AwaitAck(reject) (Section 9.3);
- in ticket flow, an Invoke that finds the inbound invoke table full gets RetryAck (Section 9.5).

8.4 Errors

An **error** is a condition that a correct program never causes. Table 13 lists the errors. The RTL checks them with assertions, which exist only in simulation. In hardware, an error is undefined behavior. r2-sim stops the run at the first error (document 3, Section 4.9).

Table 13: The errors of the Neutrino.

Error	Cause
`Unowned`	A destination range without one owner; a key, a `wait` other than a fence, a `version` or a `vreset` at an address without an owner; a load or a store outside every memory block (Rule N2.2).
`Empty`	A destination range with `bound` 0 (Rule N2.3).
`PendingFull`	A full pending table at a local invoke or a local wait.
`VersionsFull`	A full version table at a local invoke or a `version` (Rule N6.2).
`SelfDependency`	A task that depends on its own key (Rule N7.5).
`VersionInFlight`	A version that a task in the tables holds (Rule N7.4).
`Invalid`	A PE of another node (Rule N3.1); a spawn on a Neutrino that is not a direct child, or on a child that runs a program; an untracked task with a PE of another node, with the flag `versioned`, or without a PE (Rules N3.9 and N3.10); the key 0 in `deps` (Rule N3.11); a join or a spawn outside the sync-only space (Rule N2.5); a `version` of the range of another Neutrino; more remote keys than the outbound dep table holds (Rule N6.3); a forwarded descriptor of more than 4 beats (Rule N4.3); a matmul or vector task on the linear range of a hash link (document 2, Rule R3.6).
`Unsupported`	A spawn whose `done` range belongs to another Neutrino (Rule N3.4).
`Race`	A load of a range that an issued task writes (Rule N5.2).

8.5 Program lifecycle

A Neutrino runs at most one program at a time. A program starts in one of two ways: the host starts it (the program of the device), or the parent sends Start (Section 9.7). At the start the Neutrino clears its version table. The start does not change the other tables: tasks of an earlier program that are still in flight continue.

When the program returns, the Neutrino does these steps:

1. It waits until every untracked task of the program is done, as a fence does (Rule N3.11).
2. The done unversioned entries retire. The unversioned entries that are not done retire at their completion.
3. If a parent started the program, the Neutrino sends Return to the parent.

The version table stays until the next start.

Rule N8.1 (wait before the return). The return of a program waits for the untracked tasks of the program, as a fence does. It does not wait for the tracked tasks. A program whose parent depends on the outputs of its tracked tasks must wait for those outputs before it returns.

Note. A Neutrino whose node only owns data, for example a memory owner that runs no program, never clears its version table by a start. The program that writes its ranges must free the entries with `vreset` (a VersionReset message from a remote program) to keep them within the capacity (Rule N6.2).

9 Protocols between Neutrinos

Three protocols carry the traffic between Neutrinos. Each protocol is a **protocol class** with its own channel on the control buses:

- the *dependency* class: Await, AwaitAck, Resolve (Section 9.3);
- the *invoke* class: Invoke, VersionReset, and in ticket flow InvokeAck, RetryAck, Reject, PCrdGrant (Sections 9.4 to 9.6);
- the *hierarchy* class: Start, Return (Section 9.7).

9.1 Messages

A message is 1 to 4 beats of `msg_bytes` (64 B by default). An Invoke is $\lceil bytes/\texttt{msg_bytes} \rceil$ beats, with the size of its descriptor from Section 4.3. Every other message is 1 beat.

9.2 Channels and stages

A message travels on the control path from its sender to its receiver: the shortest path over the control buses and control links (document 2, Section 3.4). It starts at the control attachment of the sender and ends at that of the receiver. Each block of the path with a bandwidth, a direction of a link or a port of a crossbar, is a **control hop**. A block without a bandwidth adds its latency and nothing else.

The **channel** of a class from a sender to a receiver has these stages (Figure 5):

- the **source queue** of the class at the sender. For the invoke class it is the outbound invoke table, with `outbound_invokes` entries. For the other classes it has no limit: the outbound

Table 14: The messages between Neutrinos.

Message	Class	Direction	Content	Beats
Await	dependency	requester to owner	The requester, its request ID, the key.	1
AwaitAck	dependency	owner to requester	The request ID; registered, done or reject.	1
Resolve	dependency	owner to requester	The request ID: the key is done.	1
Invoke	invoke	requester to owner	The descriptor.	1 to 4
VersionReset	invoke	requester to owner	A base address.	1
InvokeAck	invoke	owner to requester	Ticket flow: the oldest request is accepted.	1
RetryAck	invoke	owner to requester	Ticket flow: the inbound invoke table was full; the requester holds a ticket.	1
Reject	invoke	owner to requester	Ticket flow: a request without a ticket while the bit of the requester is set.	1
PCrdGrant	invoke	owner to requester	Ticket flow: an inbound entry is reserved for the oldest request to replay.	1
Start	hierarchy	parent to child	The spawn task, the program.	1
Return	hierarchy	child to parent	The spawn task.	1

dep table and the inbound ref table limit the requests;

- one **stage** at the end of each control hop, per class, with `hop_queue` entries (1 by default). It is a pipeline register, not a buffer to size;
- the receiver. For an Invoke under ready/valid flow, the receiver is the inbound invoke table.

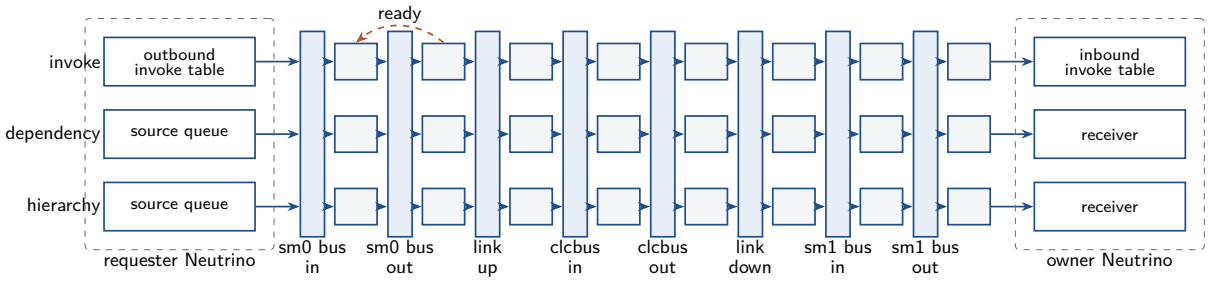


Figure 5: The channels from one SM to another SM of the same cluster: per class a source queue, one stage after each control hop, and the receiver. The three classes share the control hops and have separate stages.

The stages use **ready/valid** flow control:

1. A message at the head of a stage moves to the next stage only when the next stage has a free entry. It takes that entry at the start of its transmission.
2. The transmission moves the *beats* · `msg_bytes` bytes of the message over the control hop, one transfer per cycle. A transfer carries the smaller of the transfer width of the sender (`beat_bytes`) and that of the control hop (document 2, Section 2.3). Then the message takes the latency of the hop to the next stage. The entry that it leaves frees at the end of the transmission.
3. Before the first control hop, the message takes the latency of the blocks without a bandwidth before it.
4. At the end of the last control hop, an Invoke under ready/valid flow needs a free entry of the inbound invoke table. It reserves the entry when it leaves the last stage. Other messages need no entry.
5. The message arrives when it leaves the last stage into the receiver.

Stages never reorder messages. A message at the head of a stage that cannot move blocks the messages behind it in that stage, also the messages to other receivers: **head-of-line blocking**. Stages exist per control hop and class, not per destination. When an entry of a stage frees, the stages that wait for it move on in the order in which they blocked. Backpressure therefore reaches the source queue and, for the invoke class, the program.

Example. In the 2p configuration a message from `root.gpc0.sm0` to `root.gpc0.sm1` crosses eight control hops: two ports of the control bus of `sm0` (from its Neutrino, to its link), the link of `sm0` up, the ports of `sm0` and of `sm1` on the crossbar `clcbus`, the link of `sm1` down, and two ports of the control bus of `sm1`. Each control hop has one stage per class. The fabric is 8 B wide, so one message beat of 64 B takes 8 cycles on each hop.

9.3 Dependency protocol

A requester A has a remote key: a dependency of a task that A owns, or a `wait` of the program of A (Rule N3.5). The owner of the key is B. Figure 6 shows the three cases.

1. A opens an entry in its outbound dep table and sends Await with the key to B. The consumer counts the key as one in its depcount, or the program waits.
2. B looks the key up (Section 7.4). B rejects the request with AwaitAck(reject) in two cases: the key is not done and the inbound ref table is full, or the lookup needs a new pending entry and the pending table is full. A sends the same Await again after `t_retry` cycles.
3. When the key is done, B answers AwaitAck(done). A clears the dependency and frees its outbound dep entry.
4. When the key is not done, B records an inbound ref on the entries of the lookup (task entries, or a new pending entry) and answers AwaitAck(registered). The owner counts nothing else for the remote reader.
5. The invoke of the producer at B adopts a pending entry with its inbound refs. When all entries of an inbound ref complete, B sends Resolve to A, and the inbound ref frees. A clears the dependency and frees its outbound dep entry.

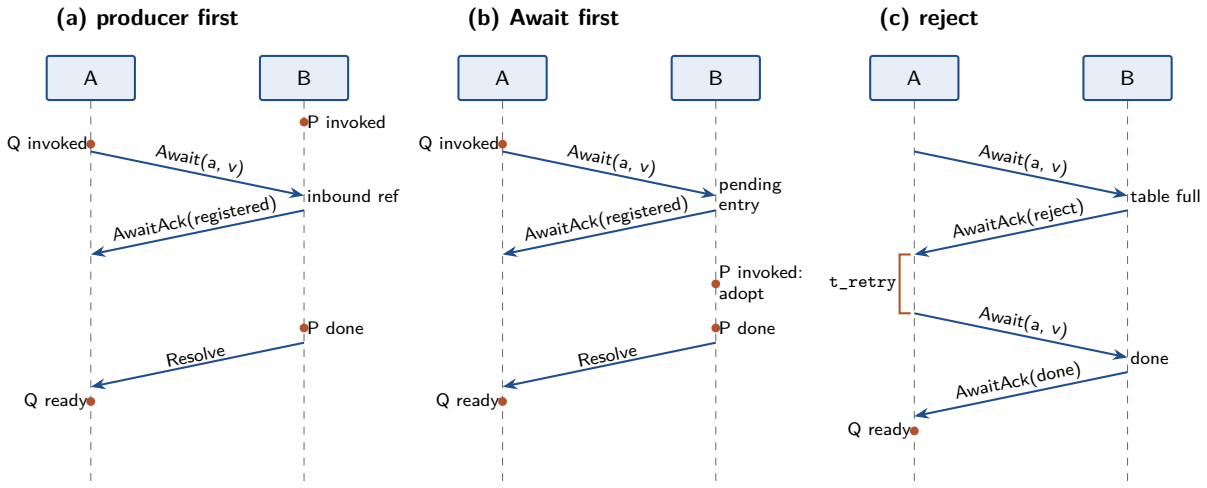


Figure 6: The dependency protocol. A consumer Q at A depends on the key (a, v) of a producer P at B. (a) P is in the tables of B. (b) The Await reaches B before the invoke of P: a pending entry waits, and the invoke of P adopts it. (c) B has no free entry: it rejects, and A sends the Await again after `t_retry`.

The dependency channel and the invoke channel are independent. An Await can therefore reach the owner before the Invoke of the producer, also when one requester sends both. The pending

table covers this case: the early Await costs a pending entry, not a scoreboard entry.

9.4 Invoke protocol

A forwarded invoke (Rule N3.3) uses the invoke channel. Under ready/valid flow, the default, it has these steps:

1. The requester puts the Invoke into its outbound invoke table. The program continues. When the table is full, the program stalls.
2. The message moves stage by stage (Section 9.2).
3. At the owner it takes a reserved entry of the inbound invoke table.
4. The task waits in the inbound invoke table for its admission (Section 6.2). At admission the owner assigns the version and resolves the dependencies: local keys in its own tables, other keys with the dependency protocol, with the owner as the requester.

There is no acknowledgement, no reject and no credit. Nothing stays at the requester after the message leaves the outbound invoke table: the message carries the descriptor.

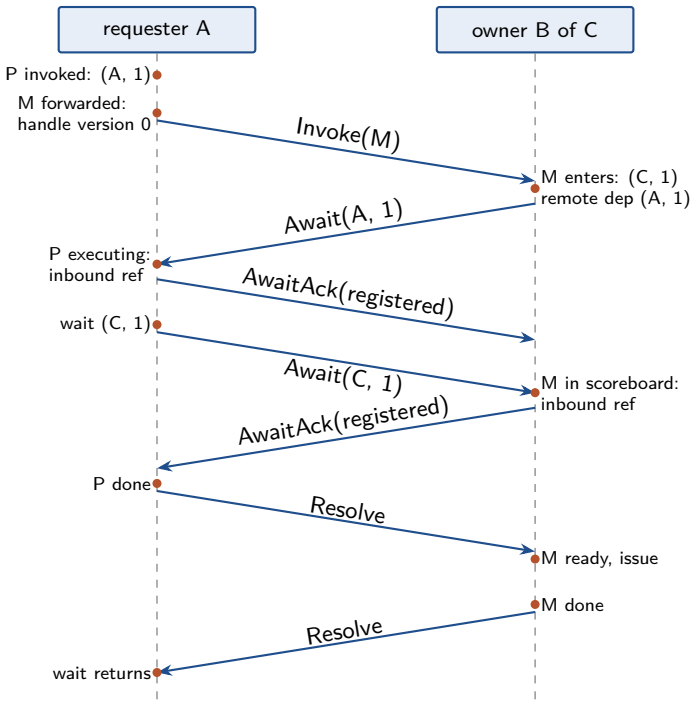


Figure 7: A forwarded invoke with a dependency back on the requester. A writes range A with task P, forwards task M (destination range C at B, dependency $(A, 1)$) and waits for $(C, 1)$.

Example. Figure 7 has these values: two SMs of one cluster, no descriptor costs, `c_invoke` 8, a matmul of 192 cycles. P is invoked at cycle 0 and runs from 8 to 200; it completes at 204. The forward of M returns at 16 with version 0. The Invoke (80 B, 2 beats) enters the scoreboard of B at 48 with version 1, and its Await for $(A, 1)$ reaches A at 74, while P runs. The Resolve of $(A, 1)$ leaves A at 204 and arrives at 230; M runs from 230 to 422 and completes at 426. The Resolve of $(C, 1)$ reaches the wait of A at 452. The example has 7 messages: the Invoke, 2 Await, 2 AwaitAck(registered) and 2 Resolve.

9.5 Ticket flow

Ticket flow is a comparison configuration (`flow: ticket`), in the style of CHI. It is equal to ready/valid flow except at a full inbound invoke table. The invoke channel carries answers from the owner to the requester. The requester keeps each request (Invoke or VersionReset) in its outbound invoke table until its InvokeAck or its RetryAck.

The owner answers each request in this order of rules:

1. A replay with a ticket takes its reserved entry: InvokeAck. After the last replay of the requester is accepted, the bit of the requester clears.
2. A request without a ticket from a requester whose bit is set: Reject.
3. An Invoke or a VersionReset that finds the inbound invoke table full: RetryAck, and the owner sets the bit of the requester.
4. Every other request: InvokeAck.

While the inbound invoke table has free entries, the owner grants them. It reserves an entry for the next requester with a set bit and no grant in flight, round-robin after the requester of the last grant. Then it sends PCrdGrant to that requester.

The requester acts on the answers:

1. InvokeAck: the oldest request held for that owner is accepted, and its entry frees.
2. RetryAck for request X: every request before X was accepted, and every request after X that is already sent comes back rejected. The requester marks X and those requests for replay. While it holds a ticket at any owner, the requester sends only replays from its outbound invoke table. This stop at every owner is a choice of the implementation: the outbound invoke table is one queue with one head. The protocol needs the stop only at the owner that gave the ticket.
3. Reject: the request is already marked for replay.
4. PCrdGrant: the requester sends its oldest marked request again, with the ticket, before every other request. After the last replay the ticket ends.

The owner accepts the requests of each requester in program order. The state is $N - 1$ bits per owner, at most one ticket per requester and owner, and the outbound entries until their acknowledgement. The cost is one InvokeAck per accepted request and one Reject per refused request.

Example. In this example on the 2p configuration, the device forwards 40 joins round-robin to two SMs. The first SM has 2 inbound invoke entries, 4 execution entries and `c_complete` 100. Under ready/valid flow the device sends 40 Invoke, and the example ends at cycle 2397. Under ticket flow it sends 45 Invoke (5 replays), 40 InvokeAck, 2 RetryAck, 3 Reject and 5 PCrdGrant, and it ends at cycle 2833. Each answer of the owner is a message of one beat, which takes 8 cycles on each hop of the control fabric of 8 B.

9.6 VersionReset

A `vreset` of a base address of another Neutrino sends VersionReset to the owner in the invoke channel. It takes an entry of the outbound invoke table and stays in program order with the forwarded invokes of the program to that owner. At the owner it takes an entry of the inbound invoke table, as an Invoke does. The owner frees the version table entry when the message reaches the head of the table: after the earlier invokes of the same requester have their versions. An absent entry is not an error. So all the messages of the invoke protocol from one requester take effect in program order.

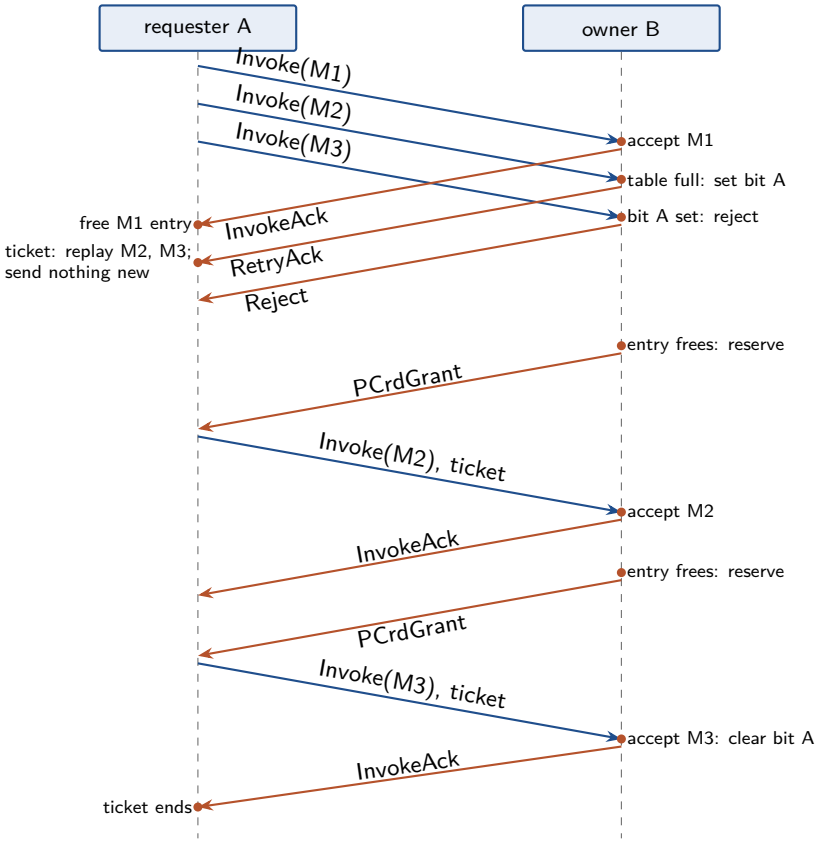


Figure 8: Ticket flow. B has one free inbound invoke entry. M1 takes it, M2 gets RetryAck, and M3, already in flight, gets Reject. Each PCrdGrant reserves a freed entry for the next replay.

Rule N9.1 (reset after the last version). A program that resets a base address and then writes it again must first wait for the last version that it wrote. Otherwise the new invoke can get a version that a task in the tables still holds (Rule N7.4).

9.7 Hierarchy protocol

The `spawn` instruction makes a spawn task in the tables of the parent (Section 5.3). Figure 9 shows the protocol.

1. When the spawn task issues, the parent sends Start with the program to the child.
2. At arrival the child must run no program; otherwise this is the error `Invalid`. The child clears its version table and starts the program.
3. When the program returns, the child sends Return to the parent (Section 8.5).
4. The parent charges `c_complete` and completes the spawn task. The waiters of `done` wake.

Example. In this example, a cluster spawns a program of 100 cycles on its first SM. The spawn costs 16 cycles (no descriptor costs): the task issues at 16. Start arrives at 30, the child runs from 30 to 130, Return arrives at 144, and the spawn task completes at 148.

9.8 Ordering

Rule N9.2 (ordered channels). The messages of one class from one sender to one receiver arrive in the order in which the sender sent them. The path is fixed for each pair of Neutrinos,

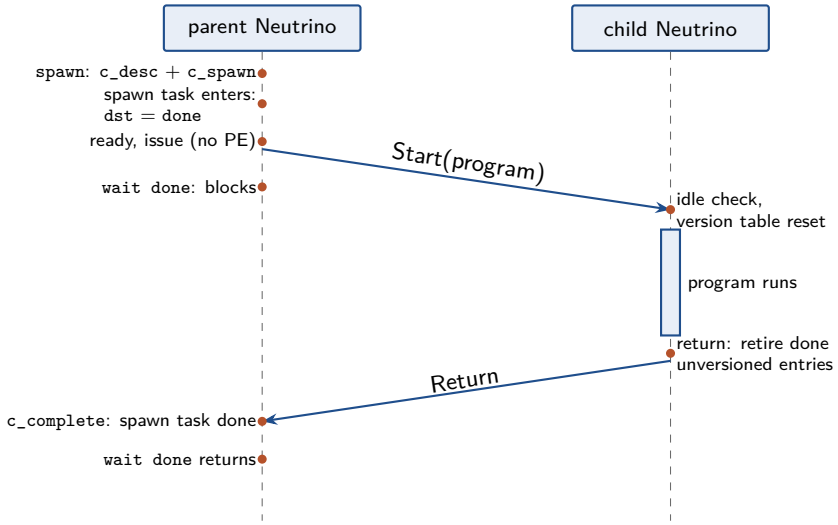


Figure 9: The hierarchy protocol. The spawn task holds the `done` range of the parent until the child program returns.

and the stages are first in, first out.

Rule N9.3 (ordered admission). An owner admits the forwarded invokes of one requester in the order of arrival, and so in program order (Rule N3.8). Under ready/valid flow the inbound invoke table is first in, first out. Under ticket flow the owner accepts the replays of a requester in order and refuses its later requests until the replays are accepted.

These orders do not exist:

- between two classes: an Await can pass an Invoke, and a Resolve can pass a Return;
- between two requesters of one owner;
- between the arrival of a VersionReset and the admission of the earlier invokes that wait in the inbound invoke table (Rule N9.1).

Two consequences follow from Rule N9.2. An AwaitAck(registered) reaches the requester before the Resolve of the same request. A requester can infer the versions of its forwarded writes (Rule N3.7).

9.9 Progress

The protocols make progress for these reasons:

- The dependency channel and the hierarchy channel are independent of the invoke channel. Await, AwaitAck, Resolve, Start and Return always drain.
- An owner drains its inbound invoke table. Admission needs scoreboard entries and issue headroom, which completions free. It also needs pending entries and version table entries, which free when the programs keep their keys and their versioned ranges within the capacities (Rule N6.2).
- A consumer never blocks its own producer. The invokes of one requester arrive in order, and an early Await of a third Neutrino costs a pending entry, not a scoreboard entry.

A program that breaks the rules of this document can deadlock. Document 3 gives how r2-sim detects a deadlock (Section 4.9).

10 Sample programs

The listings of this section are assembly text (Section 5.8) that r2-sim printed for example programs. Document 3 gives the runs that print them (Section 13.4). Marks `...` show lines that are left out. The listings of the `kernels` program of r2-sim leave out the label of each task (`label=<n>`, Section 4.1).

10.1 Layernorm on two SMs

This example runs a GPU-style kernel as a grid of CTAs on the SMs of the RTX PRO 6000 reference configuration (document 2, Section 2.5). It comes from a program of r2-sim for the comparison with the card (document 3, Section 13.3.2). The kernel is a layernorm of 4 rows of 1024 bf16 elements, one row per CTA, on two SMs: `root.gpc0.sm0` and `root.gpc1.sm0`. CTA i runs on SM i mod 2. The input x is at `root.hbm+0x0` and the output y at `root.hbm+0x2000`. The invoke costs are 0 in this example, so the invokes of one CTA have the same cycle.

Listing 2: Layernorm: the program of the device.

```
.program root kernels    ; started by the host at cycle 0
    cache_flush    ; t=0
    mark "ln start"    ; t=0
    spawn child=root.gpc0 done=root.sync+0x0:0x200 deps=[]    ; t=0
    spawn child=root.gpc1 done=root.sync+0x200:0x200 deps=[]    ; t=16
    wait root.sync+0x0    ; t=32
    wait root.sync+0x200    ; t=2125
    mark "ln end"    ; t=2141
```

The device program empties the modeled caches (`cache_flush`) and marks the start of the kernel (`mark`). These two instructions exist only in r2-sim (document 3, Section 4.2). The program spawns one program on each cluster. Each spawn has a `done` range of 512 B in the sync-only space of the device; the second spawn comes `c_spawn` = 16 cycles after the first. The two unversioned waits on the `done` ranges end when the cluster programs return.

Listing 3: Layernorm: the program of cluster `root.gpc0`.

```
.program root.gpc0 ln    ; parent root, started at cycle 75
    spawn child=root.gpc0.sm0 done=root.gpc0.sync+0x0:0x200 deps=[]    ; t=75
    wait root.gpc0.sync+0x0    ; t=91
```

The cluster program starts at cycle 75: the Start crossed the control link of the cluster (54 cycles) and the crossbars. It spawns the program of its SM and waits for it.

Listing 4: Layernorm: the program of SM `root.gpc0.sm0`.

```
.program root.gpc0.sm0 ln    ; parent root.gpc0, started at cycle 122
    dma dst=root.gpc0.sm0.smem+0x0:0x800@v1 pe=root.gpc0.sm0.dma deps=[] from=root.
    ↪ hbm+0x0 to=root.gpc0.sm0.smem+0x0 bytes=2048 tile=2048    ; t=122
    vec dst=root.gpc0.sm0.smem+0x800:0x800@v1 pe=root.gpc0.sm0.muon deps=[root.gpc0.
    ↪ sm0.smem+0x0@v1] ops=8192 in=2048 out=2048 src=root.gpc0.sm0.smem+0x0    ; t
    ↪ =122
    dma dst=root.gpc0.sm0.sync+0x0:0x200@v1 pe=root.gpc0.sm0.dma deps=[root.gpc0.sm0.
    ↪ smem+0x800@v1] from=root.gpc0.sm0.smem+0x800 to=root.hbm+0x2000 bytes=2048
    ↪ tile=2048    ; t=122
    wait root.gpc0.sm0.smem+0x800@v1    ; t=122
    dma dst=root.gpc0.sm0.smem+0x0:0x800@v2 pe=root.gpc0.sm0.dma deps=[] from=root.
    ↪ hbm+0x1000 to=root.gpc0.sm0.smem+0x0 bytes=2048 tile=2048    ; t=838
```

```
    vec dst=root.gpc0.sm0.smem+0x1000:0x800@v1 pe=root.gpc0.sm0.muon deps=[root.gpc0.
    ↪ sm0.smem+0x0@v2] ops=8192 in=2048 out=2048 src=root.gpc0.sm0.smem+0x0     ; t
    ↪ =838
    dma dst=root.gpc0.sm0.sync+0x200:0x200@v1 pe=root.gpc0.sm0.dma deps=[root.gpc0.
    ↪ sm0.smem+0x1000@v1] from=root.gpc0.sm0.smem+0x1000 to=root.hbm+0x3000 bytes
    ↪ =2048 tile=2048     ; t=838
    wait root.gpc0.sm0.sync+0x0@v1     ; t=838
    wait root.gpc0.sm0.sync+0x200@v1     ; t=1311
```

Line by line:

1. `dma`: CTA 0 copies row 0 of x (2 KiB) into the input buffer at `smem+0x0`, version 1. The engine of the SM runs it: the destination range is in the SMEM of the SM.
2. `vec`: the layernorm of the row on the Muon, 1024×8 lane operations. It reads the input buffer and writes the output buffer at `smem+0x800`. It depends on the DMA.
3. `dma`: the store of the row to `root.hbm+0x2000`. Its destination range is the store record `sync+0x0` (see below). It depends on the vector task.
4. `wait`: the SM can take the next CTA when the compute of CTA 0 is done. The wait also orders the next write of the input buffer after its reader (Rule N7.1).
5. `dma`: CTA 2 copies row 2 into the input buffer, version 2. It needs no dependency: the program waited for the reader.
6. `vec`: the layernorm into the second output buffer at `smem+0x1000`. Two output buffers let a store run while the next CTA computes.
7. `dma`: the store of row 2, with the store record `sync+0x200`.

8–9. `wait`: the program waits for both stores before it returns (Rule N8.1).

Note. The destination range of a store task is a store record: a range of 512 B in the sync-only space of the SM. The engine of the SM writes `root.hbm`, which the device owns. The `kernels` program of r2-sim uses store records for the stores of GPU kernels (document 3, Section 13.3.2). In a Radiance 2 program the store is an untracked task of the SM (Section 3.1), or a pull by the device (Section 10.6).

10.2 GEMM on one SM

This kernel computes $C = AB$ with $m = n = 128$ and $k = 64$, bf16, as one CTA of 128×128 on `root.gpc0.sm0`: two k-tiles of 32, two stages of SMEM buffers, and four warps of 64×64. The SM has four tensor units, `atlas0` to `atlas3`; warp w runs on unit w. A is at `root.hbm+0x0`, B at `root.hbm+0x4000` and C at `root.hbm+0x8000`. The programs of the device and the cluster are those of Section 10.1 with one cluster and one SM.

Listing 5: GEMM: the program of SM `root.gpc0.sm0`.

```
.program root.gpc0.sm0 g     ; parent root.gpc0, started at cycle 122
    dma dst=root.gpc0.sm0.smem+0x0:0x2000@v1 pe=root.gpc0.sm0.dma deps=[] from=root.hbm+0x0 to=root.gpc0.
     ↪ sm0.smem+0x0 bytes=8192 tile=8192     ; t=122
    dma dst=root.gpc0.sm0.smem+0x2000:0x2000@v1 pe=root.gpc0.sm0.dma deps=[] from=root.hbm+0x4000 to=root.
     ↪ gpc0.sm0.smem+0x2000 bytes=8192 tile=8192     ; t=122
    mm dst=root.gpc0.sm0.smem+0x8000:0x3f80/0x7e@v1 pe=root.gpc0.sm0.atlas0 deps=[root.gpc0.sm0.smem+0
     ↪ x0@v1,root.gpc0.sm0.smem+0x2000@v1] a=root.gpc0.sm0.smem+0x0 b=root.gpc0.sm0.smem+0x2000 lda=32
     ↪ ldb=128 ldc=128 m=64 n=64 k=32 types=bf16,bf16,bf16 acc=0 keep=1 tile=64x64     ; t=122
    mm dst=root.gpc0.sm0.smem+0xc000:0x3f80/0x7e@v1 pe=root.gpc0.sm0.atlas1 deps=[root.gpc0.sm0.smem+0
     ↪ x0@v1,root.gpc0.sm0.smem+0x2000@v1] a=root.gpc0.sm0.smem+0x1000 b=root.gpc0.sm0.smem+0x2000 lda=32
     ↪  ldb=128 ldc=128 m=64 n=64 k=32 types=bf16,bf16,bf16 acc=0 keep=1 tile=64x64     ; t=122
    mm dst=root.gpc0.sm0.smem+0x8080:0x3f80/0x7e@v1 pe=root.gpc0.sm0.atlas2 deps=[root.gpc0.sm0.smem+0
     ↪ x0@v1,root.gpc0.sm0.smem+0x2000@v1] a=root.gpc0.sm0.smem+0x0 b=root.gpc0.sm0.smem+0x2080 lda=32
     ↪ ldb=128 ldc=128 m=64 n=64 k=32 types=bf16,bf16,bf16 acc=0 keep=1 tile=64x64     ; t=122
    mm dst=root.gpc0.sm0.smem+0xc080:0x3f80/0x7e@v1 pe=root.gpc0.sm0.atlas3 deps=[root.gpc0.sm0.smem+0
     ↪ x0@v1,root.gpc0.sm0.smem+0x2000@v1] a=root.gpc0.sm0.smem+0x1000 b=root.gpc0.sm0.smem+0x2080 lda=32
     ↪  ldb=128 ldc=128 m=64 n=64 k=32 types=bf16,bf16,bf16 acc=0 keep=1 tile=64x64     ; t=122
```

```
dma dst=root.gpc0.sm0.smem+0x4000:0x2000@v1 pe=root.gpc0.sm0.dma deps=[] from=root.hbm+0x2000 to=root.
 ↪ gpc0.sm0.smem+0x4000 bytes=8192 tile=8192    ; t=122
dma dst=root.gpc0.sm0.smem+0x6000:0x2000@v1 pe=root.gpc0.sm0.dma deps=[] from=root.hbm+0x6000 to=root.
 ↪ gpc0.sm0.smem+0x6000 bytes=8192 tile=8192    ; t=122
mm dst=root.gpc0.sm0.smem+0x8000:0x3f80/0x7e@v2 pe=root.gpc0.sm0.atlas0 deps=[root.gpc0.sm0.smem+0
 ↪ x4000@v1,root.gpc0.sm0.smem+0x6000@v1,root.gpc0.sm0.smem+0x8000@v1] a=root.gpc0.sm0.smem+0x4000 b=
 ↪ root.gpc0.sm0.smem+0x6000 lda=32 ldb=128 ldc=128 m=64 n=64 k=32 types=bf16,bf16,bf16 acc=1 keep=0
 ↪ tile=64x64    ; t=122
mm dst=root.gpc0.sm0.smem+0xc000:0x3f80/0x7e@v2 pe=root.gpc0.sm0.atlas1 deps=[root.gpc0.sm0.smem+0
 ↪ x4000@v1,root.gpc0.sm0.smem+0x6000@v1,root.gpc0.sm0.smem+0xc000@v1] a=root.gpc0.sm0.smem+0x5000 b=
 ↪ root.gpc0.sm0.smem+0x6000 lda=32 ldb=128 ldc=128 m=64 n=64 k=32 types=bf16,bf16,bf16 acc=1 keep=0
 ↪ tile=64x64    ; t=122
mm dst=root.gpc0.sm0.smem+0x8080:0x3f80/0x7e@v2 pe=root.gpc0.sm0.atlas2 deps=[root.gpc0.sm0.smem+0
 ↪ x4000@v1,root.gpc0.sm0.smem+0x6000@v1,root.gpc0.sm0.smem+0x8080@v1] a=root.gpc0.sm0.smem+0x4000 b=
 ↪ root.gpc0.sm0.smem+0x6080 lda=32 ldb=128 ldc=128 m=64 n=64 k=32 types=bf16,bf16,bf16 acc=1 keep=0
 ↪ tile=64x64    ; t=122
mm dst=root.gpc0.sm0.smem+0xc080:0x3f80/0x7e@v2 pe=root.gpc0.sm0.atlas3 deps=[root.gpc0.sm0.smem+0
 ↪ x4000@v1,root.gpc0.sm0.smem+0x6000@v1,root.gpc0.sm0.smem+0xc080@v1] a=root.gpc0.sm0.smem+0x5000 b=
 ↪ root.gpc0.sm0.smem+0x6080 lda=32 ldb=128 ldc=128 m=64 n=64 k=32 types=bf16,bf16,bf16 acc=1 keep=0
 ↪ tile=64x64    ; t=122
dma dst=root.gpc0.sm0.sync+0x0:0x200@v1 pe=root.gpc0.sm0.dma deps=[root.gpc0.sm0.smem+0x8000@v2,root.
 ↪ gpc0.sm0.smem+0xc000@v2,root.gpc0.sm0.smem+0x8080@v2,root.gpc0.sm0.smem+0xc080@v2] from=root.gpc0.
 ↪ sm0.smem+0x8000 to=root.hbm+0x8000 bytes=32768 tile=32768    ; t=122
wait root.gpc0.sm0.sync+0x0@v1    ; t=122
```

Line by line:

1–2. `dma`: the A tile and the B tile of k-tile 0 (8 KiB each) into the buffers of stage 0 at `smem+0x0` and `smem+0x2000`.

3–6. `mm`: one matmul per warp, $64 \times 64 \times 32$, on `atlas0` to `atlas3`. Each depends on both DMAs. Each writes version 1 of the quarter of C of its warp. The granule of this configuration is 128 B, so a row of C (256 B) is two granules. A quarter is a masked range (the example in Section 2.3): a span of 127 granules (`0x3f80` bytes) with the mask `0x7e`. Warp w writes the rows from $64(w \bmod 2)$ and the columns from $64\lfloor w/2 \rfloor$, so the bases are `smem+0x8000`, `+0xc000`, `+0x8080` and `+0xc080`. `acc=0`: the first k-tile does not read C. `keep=1`: C stays in the accumulator of the unit.

7–8. `dma`: the tiles of k-tile 1 into the buffers of stage 1 at `smem+0x4000` and `smem+0x6000`. They need no dependency: no task read these buffers before.

9–12. `mm`: the matmuls of k-tile 1. Each depends on the two DMAs of stage 1 and on the matmul of k-tile 0 of its warp (the same C handle, version 1). `acc=1` adds to C; `keep=0` writes C to SMEM. They write version 2 of each handle.

13. `dma`: the store of C (32 KiB, from `smem+0x8000`) to `root.hbm+0x8000`, after the four last matmuls, with the store record `sync+0x0`.

14. `wait`: the SM waits for the store, then returns.

Note. Each warp needs a chain of versions for its accumulation, so the part of C of each warp needs a destination range with a base of its own. The masks make these ranges exact: each range holds the quarter of C that its warp writes. The four ranges hold no common granule, and together they hold the C buffer at `smem+0x8000`.

10.3 A dependency between two SMs

This program measures the round trip of a dependency between two SMs. The device forwards four joins. Each join writes the next version of the flag range at the base of the sync-only space of an SM and depends on the last flag of the other SM.

Listing 6: Pingpong between two SMs of `root.gpc0`.

```
.program root micro    ; started by the host at cycle 0
    join dst=root.gpc0.sm0.sync+0x0:0x200@v1 deps=[] label=0    ; t=0
```

```
    join dst=root.gpc0.sm1.sync+0x0:0x200@v1 deps=[root.gpc0.sm0.sync+0x0@v1] label=1
    ↪     ; t=19
    join dst=root.gpc0.sm0.sync+0x0:0x200@v2 deps=[root.gpc0.sm1.sync+0x0@v1] label=2
    ↪     ; t=42
    join dst=root.gpc0.sm1.sync+0x0:0x200@v2 deps=[root.gpc0.sm0.sync+0x0@v2] label=3
    ↪     ; t=65
    wait root.gpc0.sm1.sync+0x0@v2    ; t=88
```

- Each `join` is a forwarded invoke: its destination range is in the sync-only space of an SM. The device infers the versions: `@v1`, then `@v2` for each flag.
- The first join costs 19 cycles and the others 23 (one dependency): the cycles 0, 19, 42 and 65 of the listing.
- The second join enters the tables of `sm1`. Its key `root.gpc0.sm0.sync+0x0@v1` belongs to `sm0`, so `sm1` sends Await to `sm0` (Rule N3.5). The device takes no part in this dependency.
- The `wait` of the device on `root.gpc0.sm1.sync+0x0@v2` is a remote wait: an Await from the device to `sm1`.

The programs send 15 messages: 4 Invoke, 4 Await, 4 AwaitAck and 3 Resolve (one key was done when its Await arrived). It ends at cycle 591.

10.4 A barrier

This program runs a join on each of three SMs, in the sync-only space of the SM, and a join on the device that depends on all of them.

Listing 7: A barrier over three SMs.

```
.program root micro    ; started by the host at cycle 0
    join dst=root.gpc0.sm0.sync+0x0:0x200@v1 deps=[] label=0    ; t=0
    join dst=root.gpc1.sm0.sync+0x0:0x200@v1 deps=[] label=1    ; t=19
    join dst=root.gpc0.sm1.sync+0x0:0x200@v1 deps=[] label=2    ; t=38
    join dst=root.sync+0x0:0x200 deps=[root.gpc0.sm0.sync+0x0@v1,root.gpc1.sm0.sync+0
    ↪ x0@v1,root.gpc0.sm1.sync+0x0@v1] label=3    ; t=57
    wait root.sync+0x0    ; t=88
```

The three joins are forwarded invokes. The fourth join writes a range of the sync-only space of the device and is unversioned. Its three keys belong to the SMs, so the device sends three Await messages. The joins on the SMs are done when the Await messages arrive: each answer is AwaitAck(done), and no Resolve follows. The final `wait` is local and unversioned.

10.5 Forwarded invokes and inferred versions

Listing 8: Four forwarded invokes on one range.

```
.program root micro    ; started by the host at cycle 0
    join dst=root.gpc0.sm0.sync+0x0:0x200@v1 deps=[] label=0    ; t=0
    join dst=root.gpc0.sm0.sync+0x0:0x200@v2 deps=[] label=1    ; t=19
    join dst=root.gpc0.sm0.sync+0x0:0x200@v3 deps=[] label=2    ; t=38
    join dst=root.gpc0.sm0.sync+0x0:0x200@v4 deps=[] label=3    ; t=57
    wait root.gpc0.sm0.sync+0x0@v4    ; t=76
```

The device forwards four joins on one range of the sync-only space of `root.gpc0.sm0`. The handles have version 0. The device infers the versions 1 to 4, because its invokes reach the SM in program order and no other Neutrino writes the range (Rule N3.7). The `wait` names version 4. The programs send 4 Invoke, 1 Await, 1 AwaitAck and 1 Resolve.

10.6 A spawn tree and pulls

The device program starts a program on each cluster, which starts a program on its SM. Each SM copies 64 KiB of its SMEM into the SMEM of the next SM of its cluster, in copies of 16 KiB, at most 2 in flight.

Listing 9: The program of the device.

```
.program root micro    ; started by the host at cycle 0
    spawn child=root.gpc0 done=root.sync+0x0:0x200 deps=[]    ; t=0
    spawn child=root.gpc1 done=root.sync+0x200:0x200 deps=[]    ; t=27
    wait root.sync+0x0    ; t=54
    wait root.sync+0x200    ; t=1387
```

Listing 10: The program of cluster `root.gpc0`.

```
.program root.gpc0 micro_gpc    ; parent root, started at cycle 87
    spawn child=root.gpc0.sm0 done=root.gpc0.sync+0x0:0x200 deps=[]    ; t=87
    wait root.gpc0.sync+0x0    ; t=114
```

Listing 11: The program of SM `root.gpc0.sm0`.

```
.program root.gpc0.sm0 micro_sm    ; parent root.gpc0, started at cycle 164
    dma dst=root.gpc0.sm1.smem+0x80000:0x4000@v1 pe=root.gpc0.sm1.dma deps=[] from=root.gpc0.sm0.smem+0x0
     ↪ to=root.gpc0.sm1.smem+0x80000 bytes=16384 tile=16384 label=0    ; t=164
    dma dst=root.gpc0.sm1.smem+0x84000:0x4000@v1 pe=root.gpc0.sm1.dma deps=[] from=root.gpc0.sm0.smem+0
     ↪ x4000 to=root.gpc0.sm1.smem+0x84000 bytes=16384 tile=16384 label=2    ; t=187
    wait root.gpc0.sm1.smem+0x80000@v1    ; t=210
    dma dst=root.gpc0.sm1.smem+0x80000:0x4000@v2 pe=root.gpc0.sm1.dma deps=[] from=root.gpc0.sm0.smem+0x0
     ↪ to=root.gpc0.sm1.smem+0x80000 bytes=16384 tile=16384 label=4    ; t=617
    wait root.gpc0.sm1.smem+0x84000@v1    ; t=640
    dma dst=root.gpc0.sm1.smem+0x84000:0x4000@v2 pe=root.gpc0.sm1.dma deps=[] from=root.gpc0.sm0.smem+0
     ↪ x4000 to=root.gpc0.sm1.smem+0x84000 bytes=16384 tile=16384 label=6    ; t=816
    wait root.gpc0.sm1.smem+0x80000@v2    ; t=839
    wait root.gpc0.sm1.smem+0x84000@v2    ; t=1070
```

- The device spawns the two cluster programs 27 cycles apart: the descriptor of a spawn (24 B) costs 11 cycles and `c_spawn` 16.
- `root.gpc0` starts at cycle 87 and `root.gpc0.sm0` at cycle 164: the Start messages cross the control fabric, 8 B wide.
- Each `dma` of the SM is a pull. Its destination range is in the SMEM of `root.gpc0.sm1`. So the descriptor names the engine of `sm1`, and the invoke is forwarded (Rule N3.2). The engine of `sm1` reads the SMEM of `sm0`.
- The two buffers at `+0x80000` and `+0x84000` each get versions 1 and 2. Before it reuses a buffer, the SM waits for the earlier version of the buffer with a remote wait.
- The SM program waits for its last versions before it returns (Rule N8.1).

The programs send 4 Start, 4 Return, 8 Invoke, 8 Await, 8 AwaitAck and 6 Resolve (two keys were done when their Await arrived).

10.7 Data-dependent control and version resets

This program runs the action expert of a mixture-of-experts model. Its one driver runs on `root.gpc0.sm0`. Listing 12 is an excerpt of one layer. The labels are left out.

Listing 12: Excerpt of the driver of the mixture-of-experts layer (labels left out).

```
.program root.gpc0.sm0 adamoe    ; started by the host at cycle 0
    vec dst=root.gpc0.sm0.smem+0x19800:0x19800@v? pe=root.gpc0.sm0.muon deps=[] ops=208896 in=104448 out
     ↪ =104448 label=...    ; t=0
```

```
dma dst=root.gpc0.sm1.smem+0x0:0x19800@v1 pe=root.gpc0.sm1.dma deps=[root.gpc0.sm0.smem+0x19800@v1]
 ↪ from=root.gpc0.sm0.smem+0x19800 to=root.gpc0.sm1.smem+0x0 bytes=104448 tile=65536 label=...    ; t
 ↪ =22
dma dst=root.gpc0.sm1.smem+0x80000:0x40000@v1 pe=root.gpc0.sm1.dma deps=[] from=root.gpc0.hbm+0x0 to=
 ↪ root.gpc0.sm1.smem+0x80000 bytes=262144 tile=65536 label=...    ; t=49
dma dst=root.gpc0.sm1.smem+0xc0000:0x40000@v1 pe=root.gpc0.sm1.dma deps=[] from=root.gpc0.hbm+0x40000
 ↪ to=root.gpc0.sm1.smem+0xc0000 bytes=262144 tile=65536 label=...    ; t=72
mm dst=root.gpc0.sm1.smem+0x19800:0x3400@v1 pe=root.gpc0.sm1.atlas deps=[root.gpc0.sm1.smem+0x0@v1,
 ↪ root.gpc0.sm1.smem+0x80000@v1] a=root.gpc0.sm1.smem+0x0 b=root.gpc0.sm1.smem+0x80000 lda=1024 ldb
 ↪ =128 ldc=128 m=51 n=128 k=1024 types=bf16,bf16,bf16 acc=0 keep=0 tile=128x128 label=...    ; t=95
mm dst=root.gpc0.sm1.smem+0x1cc00:0x3400@v1 pe=root.gpc0.sm1.atlas deps=[root.gpc0.sm1.smem+0x0@v1,
 ↪ root.gpc0.sm1.smem+0xc0000@v1] a=root.gpc0.sm1.smem+0x0 b=root.gpc0.sm1.smem+0xc0000 lda=1024 ldb
 ↪ =128 ldc=128 m=51 n=128 k=1024 types=bf16,bf16,bf16 acc=0 keep=0 tile=128x128 label=...    ; t=128
join dst=root.gpc0.sm1.sync+0x0:0x200@v1 deps=[root.gpc0.sm1.smem+0x19800@v1,root.gpc0.sm1.smem+0
 ↪ x1cc00@v1] label=...    ; t=161
...
join dst=root.gpc0.sm0.sync+0x0:0x200@v? deps=[root.gpc0.sm0.smem+0x0@v14] label=...    ; t=29314
wait root.gpc0.sm0.smem+0x7f800@v1    ; t=29337
load root.gpc0.sm0.smem+0x7f800 len=51    ; t=118859
vreset root.gpc0.sm1.sync+0x0    ; t=124568
vreset root.gpc0.sm1.sync+0x200    ; t=124569
vreset root.gpc0.sm1.sync+0x400    ; t=124570
...
vreset root.gpc1.sm3.sync+0x600    ; t=124602
join dst=root.gpc0.sm0.sync+0x200:0x200@v? deps=[root.gpc0.sm1.sync+0x1400@v1,root.gpc0.sm2.sync+0
 ↪ x800@v1,root.gpc0.sm3.sync+0x800@v1,root.gpc1.sm0.sync+0xa00@v1,root.gpc1.sm1.sync+0x800@v1,root.
 ↪ gpc1.sm2.sync+0x800@v1,root.gpc1.sm3.sync+0x800@v1,root.gpc0.sm0.sync+0x0@v1] label=...    ; t
 ↪ =124609
dma dst=root.gpc0.sm1.smem+0x0:0x19800@v3 pe=root.gpc0.sm1.dma deps=[root.gpc0.sm1.sync+0x1400@v1]
 ↪ from=root.gpc0.sm0.smem+0x33000 to=root.gpc0.sm1.smem+0x0 bytes=34816 tile=34816 label=...    ; t
 ↪ =124660
dma dst=root.gpc0.sm1.smem+0xc0000:0x40000@v30 pe=root.gpc0.sm1.dma deps=[root.gpc0.sm1.sync+0x1400@v1
 ↪ ] from=root.gpc0.hbm+0x500000 to=root.gpc0.sm1.smem+0xc0000 bytes=262144 tile=65536 label=...    ;
 ↪  t=124687
dma dst=root.gpc0.sm1.smem+0x80000:0x40000@v31 pe=root.gpc0.sm1.dma deps=[root.gpc0.sm1.sync+0x1400@v1
 ↪ ] from=root.gpc0.hbm+0x8c0000 to=root.gpc0.sm1.smem+0x80000 bytes=262144 tile=65536 label=...    ;
 ↪  t=124714
mm dst=root.gpc0.sm1.smem+0x4d600:0x3400@v4 pe=root.gpc0.sm1.atlas deps=[root.gpc0.sm1.smem+0x0@v3,
 ↪ root.gpc0.sm1.smem+0xc0000@v30,root.gpc0.sm1.sync+0x1400@v1] a=root.gpc0.sm1.smem+0x0 b=root.gpc0.
 ↪ sm1.smem+0xc0000 lda=1024 ldb=128 ldc=128 m=17 n=128 k=1024 types=bf16,bf16,bf16 acc=0 keep=0 tile
 ↪ =128x128 label=...    ; t=124741
dma dst=root.gpc0.sm1.smem+0xc0000:0x40000@v31 pe=root.gpc0.sm1.dma deps=[root.gpc0.sm1.smem+0
 ↪ x4d600@v4,root.gpc0.sm1.smem+0xc0000@v30] from=root.gpc0.hbm+0xc80000 to=root.gpc0.sm1.smem+0
 ↪ xc0000 bytes=262144 tile=65536 label=...    ; t=124778
...
```

- The first `vec` is a local invoke: its `dst` is in the SMEM of the driver. Its version comes back in the handle, so the listing shows `@v?`.
- The first `dma` is a forwarded pull: `sm1` copies the output of that `vec` from the SMEM of `sm0`. Its key `root.gpc0.sm0.smem+0x19800@v1` is remote at `sm1`, which sends Await to `sm0`.
- The `join` on `root.gpc0.sm1.sync+0x0` is a signal of `sm1`: it collects the two matmul outputs of `sm1`.
- `wait` and `load`: the driver waits for the output of the router and loads it (51 bytes, one expert ID per token). The program chooses the next tasks from these values.
- `vreset`: the driver frees the signal ranges of the SMs with VersionReset messages, after the barrier of the window (Rule N9.1).
- The last `dma` writes version 31 of a weight buffer. It depends on the last reader of version 30 and on version 30 itself (Rule N7.1).

11 Configuration parameters

Each Neutrino has the parameters of this section. The global parameters are the same for all Neutrinos. Document 3 gives the keys of these parameters in a configuration file of r2-sim (Section 10.4).

11.1 Tables

Table 15: The capacities of the tables. "2p" is the value of the 2p configuration (device, cluster and SM alike unless noted).

Parameter	Meaning	Default	2p
`scoreboard`	Scoreboard entries.	64	128
`execution`	Execution table entries.	64	256
`pending`	Pending table entries.	16	256
`versions`	Version table entries.	256	256; cluster 1024
`inbound_refs`	Inbound ref table entries.	16	128
`inbound_invokes`	Inbound invoke table entries, in messages.	64	64
`outbound_deps`	Outbound dep table entries.	16	128
`outbound_invokes`	Outbound invoke table entries, in messages.	16	32
`issue_headroom`	Execution entries kept for ready tasks (Rule N6.1).	PEs of the node	default

A configuration must give `inbound_invokes` ≥ 1, `outbound_invokes` ≥ 1, `scoreboard` ≥ 1, and `execution` $>$ `issue_headroom`.

Example. In the RTX PRO 6000 reference configuration, the device owns the L2 and the DRAM. Its tables are larger: scoreboard 1024, pending and execution 4096, inbound refs 1024, inbound invokes 256, outbound deps 256, outbound invokes 64. Its version table has 1 048 576 entries, so that it is never full: the card has no version table.

11.2 Costs and rates

The costs and their defaults are `c_invoke` 8, `c_base` 8, `c_dep` 3, `c_word` 1, `c_spawn` 16, `c_wait` 2, `c_version` 1, `c_vreset` 1, `c_load` 1, `c_store` 1, `c_complete` 4, `c_resolve` 0 and `c_plan` 0, in cycles. Table 6 gives what each cost is charged for. The other parameters of a Neutrino are in Table 16.

Table 16: The rates and the other parameters of a Neutrino.

Parameter	Meaning	Default
`scalar_rate`	Rate of the scalar resource (cycles of cost per cycle).	1
`complete_rate`	Rate of the completion port.	1
`t_retry`	Cycles before a rejected Await is sent again.	50
`beat_bytes`	The transfer width of the Neutrino as a master: loads, stores and messages.	64
`master`	The buses on which the Neutrino is a master.	the first control bus and every data bus of the node

Note. The defaults must not make the Neutrino the bottleneck.

Table 17: The global parameters of the Neutrinos.

Parameter	Meaning	Default
`neutrino_granule`	The granule: the unit of the addresses of a Neutrino (Section 2.3). It must be a power of two.	none
address fields	The prefix fields: a width in bits per level below the device (Section 2.1).	none
offset width	The width of the offset.	none
`msg_bytes`	The message beat: the bytes of one beat of a message.	64
flow control	Ready/valid flow or ticket flow in the invoke channel (Sections 9.4 and 9.5).	ready/valid
`hop_queue`	The stage depth: the messages of one class that a stage holds (at least 1).	1

11.3 Global parameters

Example. The 2p configuration has a granule of 512 B, a cluster field of 2 bits, an SM field of 3 bits, an offset of 33 bits, a message beat of 64 B, ready/valid flow and a stage depth of 1.